PDFL: A Privacy-Enhancing and Robust Poisoning Defense Federated Learning Scheme

Gang Wang, Huiwen Wu, Qingming Li, Ziyao Liu, Jun Zhao, *Member, IEEE*, Kwok-Yan Lam, *Senior Member, IEEE*, Qingkuan Dong, Yao Yang, and Li Zhou

***Abstract*—This paper addresses the security and privacy issues associated with the global models in Federated Learning by proposing a new approach, called PDFL, which tackles the challenges of poisoning attacks and privacy leakage during FL training rounds. PDFL is based on secure multi-party computation and performs privacy-preserving cluster analysis on encrypted data from participants in order to identify malicious poisoning attackers. This approach involves a two-server mechanism and integrates four privacy-preserving protocols based on two-party computation (2PC): SecJudge for normalizing gradients, SecCosine for computing the cosine similarity values among gradients, SecClu for countering poisoning attacks, and SecAgg for secure aggregation by the server. These protocols are designed to achieve low computational costs, preserve client data privacy, and mitigate poisoning attacks from the potentially malicious clients. We provide a theoretical proof that our four sub-protocols and the PDFL scheme are both safe and reliable, demonstrating that PDFL can ensure the privacy and security of the participating data. Additionally, we conduct extensive simulation experiments to evaluate the accuracy, efficiency, computational overhead, and communication overhead associated with the PDFL scheme. Experimental results show the potential of the PDFL scheme in significantly enhancing the ability to identify malicious poisoning attackers in federated learning systems accurately and efficiently, hence making PDFL a promising solution for addressing privacy and security concerns in this domain.**

***Index Terms*—Privacy-preserving, federated learning, poisoning attack, secret sharing.**

I. Introduction

FEDERATED learning (FL) is an efficient distributed learning system which addresses the data island problem while ensuring the privacy of participants [1]. In the context of FL, participants train their models locally based on sensitive data, and then upload the gradients or parameters of the local models to the central server. The central server subsequently aggregates the information from all participants. Since the sensitive data always remains on the participants' local devices, this approach ensures data security and privacy.

However, due to the lack of restrictions on local participants, malicious parties can forge poisoned data and upload it to the central server to compromise the global model [2], [3]. Additionally, although federated learning has garnered significant attention and utilization in various areas, it poses risks of data privacy breaches and security vulnerabilities during the model construction process. The shared global model parameters may potentially contain sensitive contributions and other sensitive information from the participants. Furthermore, hidden features in these uploaded local parameters are likely to be exploited by untrustworthy servers or other malicious participants [4]. Adversaries [5], [6], [7] may steal gradients or parameters to reconstruct and infer the sensitive data of participants.

Data poisoning attacks are usually caused by malicious users in FL system. Malicious participants utilize data with backdoors to train the local model, thereby injecting backdoors into the global model. The work presented in [8] and [9] explores the potential of backdoor attacks by a malicious participant in vertical split learning. Reference [10] introduced the label-flipping attacks, where malicious clients manipulate labels to degrade global model accuracy.

To defend against poisoning attacks from FL clients and build a robust federated learning system, several methods have been proposed to identify malicious clients. Krum [11] proposed a Byzantine-resilient aggregation rule based on squared distances. Subsequent works [12], [13], [14], [15] proposed Byzantine-tolerant aggregation methods using cosine similarity, which achieve robustness for both IID and non-IID data. Additionally, studies [16], [17], [19], [20], [28] focus on data poisoning attack detection in non-iid data distribution scenarios. Reference [21] defenses against poisoning attacks and tends to remove outliers based on Pearson correlation coefficient, which can effectively defend against label-flipping and backdoor attacks. Research in [22] demonstrates that certain classes of server attacks can be mitigated effectively by enforcing constraints such as norm bounds on participants' updates.

Gradient data theft attacks typically involve malicious actors in the data transmission link who intercept plaintext model

Received 9 November 2024; revised 20 October 2025; accepted 15 December 2025. Date of publication 24 December 2025; date of current version 5 January 2026. This work was supported by the National Key Research and Development Program of China under Grant 2022YFB4501500 and Grant 2022YFB4501504. The associate editor coordinating the review of this article and approving it for publication was Prof. Alptekin Küpçü. *(Corresponding author: Li Zhou.)*

Gang Wang, Ziyao Liu, and Kwok-Yan Lam are with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail: 1479758491@qq.com; ziyao002@e.ntu.edu.sg; kwokyan.lam@ntu.edu.sg).

Huiwen Wu, Yao Yang, and Li Zhou are with Zhejiang Lab, Hangzhou 310000, China (e-mail: huiwenw0820@outlook.com; yangyao@zhejianglab.com; zhou.li@zhejianglab.com).

Qingming Li is with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310058, China (e-mail: qingmingli45@163.com).

Jun Zhao is with the School of Computer Science, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: junzhao@ntu.edu.sg).

Qingkuan Dong is with the State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an 710071, China (e-mail: qkdong@mail.xidian.edu.cn).

Digital Object Identifier 10.1109/TIFS.2025.3648147

1556-6021 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Anhui University. Downloaded on October 18,2026 at 03:38:23 UTC from IEEE Xplore. Restrictions apply.

parameters or gradients to reconstruct users' real local datasets. These model parameter theft attacks jeopardize the privacy and security of local user data. For instance, [23] demonstrated that adversaries could steal parameters to train a Generative Adversarial Network (GAN) capable of generating prototypical samples from private targeted training sets.

To protect privacy in FL and mitigate malicious attacks via gradients or parameters, several privacy-preserving technologies have been proposed, including Homomorphic Encryption (HE), Differential Privacy (DP), Secure Multi-Party Computation (MPC), and Trusted Execution Environment (TEE).

Homomorphic Encryption allows computations to be performed directly on encrypted data without decryption. Studies [12], [14], [21], [24], [25], [26] designed a defense strategy using two-trapdoor homomorphic encryption, which can resist encrypted model poisoning while preserving privacy. In [35] and [36], Fotohi et al. presented lightweight and secure federated deep learning models. By combining homomorphic encryption with blockchain, their framework ensures data privacy, defends against various active attacks through mechanism design, while maintaining scalability and efficiency.

Local Differential Privacy enables clients to perform a differentially private transformation to their gradients, ensuring that individual data samples within local datasets are hard to identify from the gradients. For instance, [27], [28] trained FL models in a differentially private manner, providing both privacy and certified robustness against poisoning attacks.

Multi-Party Computation (MPC) allows multiple parties to collaboratively compute a function in a protocol over their inputs while keeping those inputs private. Research [16], [29], [30], [31] has utilized MPC to secure gradient privacy, offering enhanced privacy and robust defenses against backdoor attacks in federated learning.

Trusted Execution Environment (TEE) enables the execution of code within a secure, isolated environment. The work in [32] employed TEE to protect gradient privacy, providing a privacy-enhancing and robust defense against backdoors for federated learning.

While the aforementioned schemes achieve a certain degree of privacy preservation and defense against poisoning attacks, several significant issues remain: (1) Some approaches integrate computationally expensive cryptographic operations with simplistic similarity measurements. However, the efficiency of these cryptographic operations is often inadequate for practical applications, and simple similarity measures alone are frequently insufficient for accurately identifying malicious clients. (2) Other schemes naively expose intermediate information [33], such as similarities between local updates or aggregated model results, to the server. This practice may lead to client data leakage to the server, compromising client privacy. Compared to existing methods, our scheme PDFL addresses issues (1) and (2) by constructing lightweight operators based on Beaver triple, resulting in a more secure and efficient methods.

In this paper, we consider a semi-honest threat model for both clients and servers, meaning that they strictly adhere to the protocol specifications during execution. However, we assume that malicious poisoning attackers may exist among the clients. These attackers upload poisoned parameters or gradients to the servers to compromise the global model. To strengthen the robustness of the entire federated learning system, we employ computationally efficient secure multi-party protocols to guarantee client data privacy and utilize clustering algorithms to identify malicious poisoning attackers, thereby improving the system's overall efficiency and accuracy.

To summarize, the main contributions of this work are as follows:

- We design four privacy-preserving protocols based on two-party computation (2PC): SecJudge, SecCosine, SecClu and SecAgg, which have low computational costs and ensure the client data privacy.
- We develop a privacy-preserving and Byzantine robust FL scheme, named PDFL, that defends against the poisoning attack and preserves the privacy of clients by employing two-party computation (2PC) on the server side.
- We analyze the utility of PDFL through extensive simulation experiments, which demonstrate that our scheme has high accuracy and efficiency.

The remaining parts of this paper are organized as follows. In Section II, we outline the problem definition, threat model, and goals of PDFL. Section III introduces the backgrounds, including Federated Learning, Poisoning Attacks, Secret Sharing, and Clustered Federated Learning. In Section IV, we describe the details of our PDFL. Section V analyzes the data security, algorithm complexity and functionality of our PDFL. In section VI, we conduct a series of experiments to evaluate the performance. Finally, we present our conclusions in Section VII.

II. Problem Statement

A. Problem Definition

In our scheme, there are three entities: the *Trusted Authority (TA)*, the *Clients* and the *Central Aggregation Server*. The TA distributes the parameters to clients, who use them to encrypt their local gradients before uploading. The server then aggregates these encrypted model updates.

In this model, three challenges must be addressed. First, gradients from each client may leak sensitive information about their local data. Therefore, clients must ensure the privacy of their data during this process. Second, the server itself poses a potential privacy risk. Consequently, the system must be designed to prevent the server from compromising the privacy of the clients' gradients. Finally, some malicious clients may launch poisoning attacks to undermine the learning process. A robust federated learning system must be able to defend against such attacks.

B. Threat Model and Goals

We define the threat model for our PDFL as follows.

- *Trusted Authority (TA)*: We assume the TA is fully trusted and secure against malicious attacks.
- *Clients (with Poisoning Attackers)*: We assume that the majority of clients are *honest-but-curious*, meaning they follow the protocol but may attempt to infer information

Authorized licensed use limited to: Anhui University. Downloaded on October 18,2026 at 03:38:23 UTC from IEEE Xplore. Restrictions apply.

from the messages they receive. However, a minority of clients (fewer than 50%) are *malicious* and may launch poisoning attacks to undermine the global model.

- *Servers (Honest-but-Curious)*: We assume that both servers are *honest-but-curious*. Crucially, we also assume that they do not collude with each other.

Under the above threat model, the goals of our scheme, PDFL, are to ensure the security of clients' local gradients (i.e., to prevent clients' private data from being leaked to the two servers or other clients) and to defend against poisoning attacks from clients, thereby preventing malicious ones from compromising the global model's accuracy.

The threat model in this paper considers both the rationality and applicability: In the real world, servers are generally operated by service providers (such as cloud service providers) who, motivated by incentives and responsibilities, typically follow protocol rules without actively undermining them. In contrast, clients represent users or devices that may originate from untrusted environments. In the dual-server setting, we assume the servers do not actively collude, as they are usually managed by different organizations, making collusion costly. Two-party, non-colluding architectures are already deployed in practice. One prominent example is the privacy-preserving measurement system Prio, which enables two independent servers to jointly compute aggregate statistics without learning individual user data [37]. Nevertheless, while the possibility of collusion remains and would require further technical measures (such as threshold cryptography) to mitigate, which is not the scope of this paper proposed here.

III. Preliminaries

This section introduces the fundamentals of Federated Learning, Poisoning Attack, Clustered Federated Learning and Secret Sharing. Building upon these concepts, we then construct a novel FL scheme designed to enhance the privacy robustly defense against backdoor attacks. The mathematical notations frequently used in this article are summarized in Table I.

A. Federated Learning

1) Deep Learning: Generally, a deep learning model consists of three layers: the input layer, hidden layer and output layer, which can be described as a function $f(x, \Theta) = y$, where x denotes the users' inputs, Θ is model parameters, and y is the output of the function.

2) Stochastic Gradient Descent (SGD): Considering Gradients Descent (GD), we assume that a dataset D contains T training pairs, denoted as $D = \{(x_j, y_j), j = 1, 2, \ldots, T\}$, the loss function can be described as:

$$L_f(D, \Theta) = \frac{1}{T} \sum_{\langle x_j, y_j \rangle \in D} \left(y_j - f\left(x_j, \Theta\right)\right)^2. \tag{1}$$

Different from *GD*, in the t-th iteration, *SGD* will randomly choose a subset $D^{*t} \subset D$ instead of the entire data set. The loss function in this case is:

$$L_f\left(D^{*t}, \Theta_t\right) = \frac{1}{|D^{*t}|} \sum_{\langle x_j, y_j \rangle \in D^{*t}} \left(y_j - f\left(x_j, \Theta_t\right)\right)^2. \tag{2}$$

TABLE I
Summary of Notations

Notation	Description
m	The total number of clients
Θ	The client's model parameters
α	The learning rate
η	The aggregation weight
D	Client's data set
$Add(\cdot)$	The addition of secret sharing
$Mul(\cdot)$	The multiplication of secret sharing
g_i	The gradient data of client i
u, v, z	The multipliation beaver triple
S_1, S_2	Two servers
$\langle u \rangle$, $\langle v \rangle$, $\langle z \rangle$	The secret sharing of u, v, z
$Cosine(,)$	The cosine similarity
C, c, c_1, c_2	The clusters set
a_{across}	The maximum similarity
$< \cdot >$	Dot product
$\|\| \cdot \|\|$	The euclidean norm
$a_{\text{across}}^{\text{threshold}}$	The similarity threshold
$\langle g_i \rangle_1$ and $\langle g_i \rangle_2$	The secret sharing of client i gradients
e and f	The parameters of share
η_1 and η_2	The secret sharing of learning rate

According to the above loss function, in the k-th iteration, the gradient will be:

$$g_t = \nabla L_f\left(D^{*t}, \Theta_t\right). \tag{3}$$

The model parameter is updated as follows:

$$\Theta_{t+1} = \Theta_t - \alpha \nabla L_f\left(D^{*t}, \Theta_t\right), \tag{4}$$

with α the learning rate of the client.

3) Federated Aggregation: Federated Learning enables multiple organizations to collaboratively train a global model. This process is facilitated by a central server that aggregates parameters from the clients. In the t iteration, client i calculates its gradient g_i^t, and the central server aggregates the gradients from m clients to obtain $g_{\text{agg}}^t = \sum_{i=1}^{i=m} g_i^t$, then broadcasts this result. Subsequently, each client updates the local model using the aggregated gradient:

$$\Theta_{t+1} = \Theta_t - \alpha \frac{\sum_{i=1}^{i=m} g^t}{m}. \tag{5}$$

B. Poisoning Attack

The purpose of the poisoning attack is to manipulate the specified feature space into the target class. In this paper, we focus on two kinds of poisoning attack: the backdoor attack and the label-flipping attack.

Fig. 1 illustrates the backdoor attack, the malicious attacker trains the local model using images embedded with a trigger and then uploads the poisoned parameters. The attacker's goal is to establish a strong association between the trigger and the target label, ensuring that any image containing the trigger is misclassified as the target.

Authorized licensed use limited to: Anhui University. Downloaded on October 18,2026 at 03:38:23 UTC from IEEE Xplore. Restrictions apply.

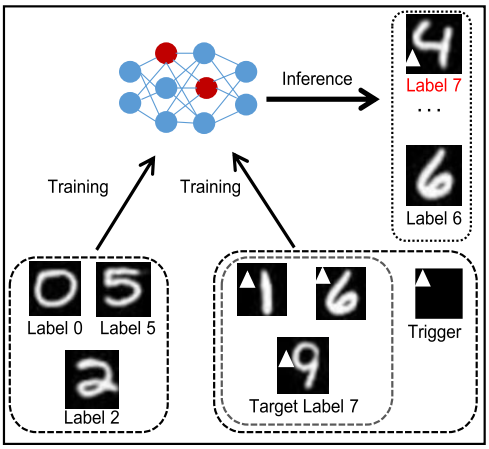


Fig. 1. Backdoor attack.

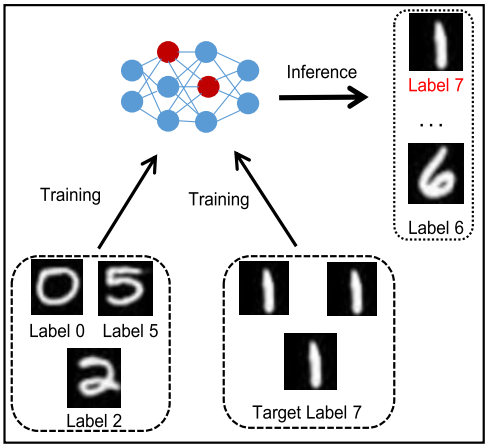


Fig. 2. Label flipping attack.

Fig. 2 illustrates the label-flipping attack. The attacker poisons the local model by training it with incorrectly labeled images and then uploads the resulting parameters. In this scenario, subsequent benign images will be misclassified as the target label. A critical aspect is that a single poisoner is sufficient to cause the final model to make incorrect predictions.

The objectives vary across different types of attacks. We define the success criteria for poisoning attacks, i.e. Attack Success Rate (ASR) and Attack Rate (AR), to enable a fair comparison of the robustness between our proposed method in Section VI-D and various defense schemes.

Definition 1 (Attack Success Rate (ASR)): The *Attack Success Rate (ASR)* is defined as the probability that a triggered test sample is classified into the target label chosen by the adversary. Formally,

$$\text{ASR} = \frac{\left|\{\, x \in \mathcal{D}_{\text{trigger}} \mid f(x) = y_{\text{target}} \,\}\right|}{\left|\mathcal{D}_{\text{trigger}}\right|}, \tag{6}$$

where $\mathcal{D}_{\text{trigger}}$ denotes the set of triggered test samples, $f(\cdot)$ is the global model, and y_{target} is the adversary's target label.

Definition 2 (Attack Rate (AR)): The *Attack Rate (AR)* is defined as the probability that the number of successful attacker to all clients. Formally,

$$\text{AR} = \frac{\left|ASR \times \mathcal{D}_{\text{trigger}}\right|}{|D|}, \tag{7}$$

where $\mathcal{D}$ denotes the set of test samples.

C. Clustered Federated Learning

In the context of a federated learning system, we assume that the majority of clients are honest. The goal of clustering is to divide the set of clients into two categories based on the distribution of the uploaded gradients [34]. The category with the larger quantity is regarded as the set of honest clients that participate in the server's aggregation operations, while the smaller category is considered as potential adversaries, whose gradients are excluded from the aggregation process. For this purpose, we introduce the cosine similarity to quantify the similarity between two clients, clients with high data similarity are clustered into the same group:

$$Cosine(i,j) = \frac{cos_{ij}}{\|g_i\|\|g_j\|} = \frac{< g_i, g_j >}{\|g_i\|\|g_j\|}, i, j \in C, \tag{8}$$

$$c_1, c_2 \leftarrow \arg \min_{c_1 \cup c_2 = c} \left(\max_{i \in c_1, j \in c_2} Cosine(i,j) \right), \tag{9}$$

$C = \{1, 2, \ldots, n\}$ is the set of all clients, and c is a subset of C, $c \subseteq C$.

$$a_{\text{across}} \leftarrow \max_{i \in c_1, j \in c_2} Cosine(i,j). \tag{10}$$

If the maximum similarity a_{across} between two clusters c_1 and c_2 falls below a certain threshold $a_{\text{cross}}^{\text{threshold}}$, the main cluster is then bipartitioned into two cluster candidates.

$$C_{\text{Benign}} \leftarrow \{\arg \max_{c \in \{c_1, c_2\}} |c|\}. \tag{11}$$

We select the category with the majority clients C_{Benign} as the set of honest participants, whose gradient parameters will be included in the subsequent cloud aggregation.

D. Secret Sharing

The secret sharing protocol ensures that all intermediate values are secret-shared between the two servers without disclosing the private values. We use fixed-point to represent the decimal values, and all arithmetic computations are performed over ring $\mathbb{Z}_{2^l}$.

1) The Addition of Secret Sharing Add(·): To add the two secret sharing values a and b, with $a = \langle a \rangle_1 + \langle a \rangle_2$ and $b = \langle b \rangle_1 + \langle b \rangle_2$. Server S_1 holds the secret sharing values $\langle a \rangle_1$ and $\langle b \rangle_1$, Server S_2 holds $\langle a \rangle_2$ and $\langle b \rangle_2$. Server S_i, $i \in [1, 2]$ can noninteractively add the shares $\langle c \rangle_i = \langle a \rangle_i + \langle b \rangle_i$. Server S_2 shares the value c_2 with Server S_1, who can subsequently reconstruct the addition of secret sharing $c = a + b = \langle c \rangle_1 + \langle c \rangle_2$.

2) The Multiplication of Secret Sharing Mul(·): To multiply two secret sharing values a and b, $a = \langle a \rangle_1 + \langle a \rangle_2$ and $b = \langle b \rangle_1 + \langle b \rangle_2$, we adopt the Beaver's pre-computed multiplication triples technique. u, v and z are multiplicative triples, where $z = uv$, $u = \langle u \rangle_1 + \langle u \rangle_2$, $v = \langle v \rangle_1 + \langle v \rangle_2$ and $z = \langle z \rangle_1 + \langle z \rangle_2$. Server S_1 holds the secret sharing values $\langle u \rangle_1$, $\langle v \rangle_1$, $\langle z \rangle_1$, Server

Authorized licensed use limited to: Anhui University. Downloaded on October 18,2026 at 03:38:23 UTC from IEEE Xplore. Restrictions apply.

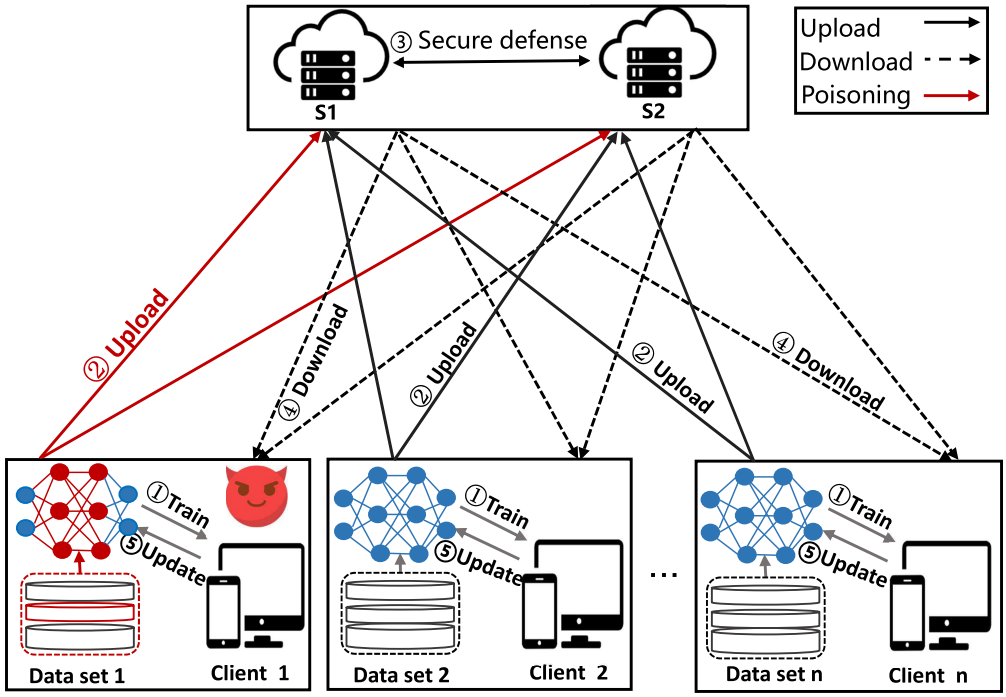


Fig. 3. Privacy-Enhancing and Robust Poisoning Defense Federated Learning Scheme.

S_2 holds the secret sharing values $\langle u\rangle_2$, $\langle v\rangle_2$, $\langle z\rangle_2$. Server S_i, $i \in [1, 2]$, locally computes

$$\langle e\rangle_i = \langle a\rangle_i - \langle u\rangle_i \quad \text{and} \quad \langle f\rangle_i = \langle b\rangle_i - \langle v\rangle_i. \tag{12}$$

Both parties reconstruct

$$e = \langle e\rangle_1 + \langle e\rangle_2 \quad \text{and} \quad f = \langle f\rangle_1 + \langle f\rangle_2. \tag{13}$$

Server S_1 calculates

$$\langle c\rangle_1 = ef + \langle a\rangle_1 f + \langle b\rangle_1 e + \langle z\rangle_1, \tag{14}$$

Server S_2 computes

$$\langle c\rangle_2 = \langle a\rangle_2 f + \langle b\rangle_2 e + \langle z\rangle_2, \tag{15}$$

and the multiplication result

$$c = ab = \langle c\rangle_1 + \langle c\rangle_2. \tag{16}$$

Compared to homomorphic encryption, secret sharing possesses significant advantages in terms of computational overhead, security and robustness. First, even for simple addition operations, homomorphic encryption involves extensive modular exponentiation, making its computational cost thousands of times slower than plaintext operations. In contrast, in additive secret sharing, participants primarily perform modular addition operations locally on their own shares, which incurs minimal overhead on the CPU and operates at speeds nearly comparable to processing plaintext data. Second, unlike homomorphic encryption, the security of secret sharing does not rely on any computational hardness assumptions. Even if an attacker possesses unlimited computational power, including quantum computers, they cannot extract any information about the original secret from a single share.

IV. Our Proposed PDFL Scheme

This section first provides an overview of PDFL, then details the construction of four subprotocols: the SecJudge subprotocol, the SecCosine subprotocol, the SecClu subprotocol and the SecAgg subprotocol. Finally, it presents the PDFL scheme and its security analysis.

A. System Model of PDFL

As shown in *Fig. 3*, the system is comprised of the *Trusted Authority (TA)*, *Clients* and the *Central Aggregation Servers*.

- *Trusted Authority (TA)*: The *TA* initializes the neural network model, generates multiplicative triples, and then broadcasts to the servers.
- *Clients*: Assume that there are m clients, each possessing a local dataset. A subset of these clients are poisoning attackers. Each client trains a local model on its own data and then employs a secret-sharing technique to split the model parameters into two shares, which are distributed to two non-colluding servers.
- *Central Aggregation Server*: The two servers, S_1 and S_2, receive encrypted data shares from the clients. They interactively run a secure aggregation protocol that is robust against model poisoning attacks, ultimately obtaining the global model, which is then returned to the clients.

B. Technical Overview of PDFL

As shown in *Fig. 4*, the PDFL includes four phases: the initialization phase, the clients train model, the servers robust aggregation, and the clients update local model.

Authorized licensed use limited to: Anhui University. Downloaded on October 18,2026 at 03:38:23 UTC from IEEE Xplore. Restrictions apply.

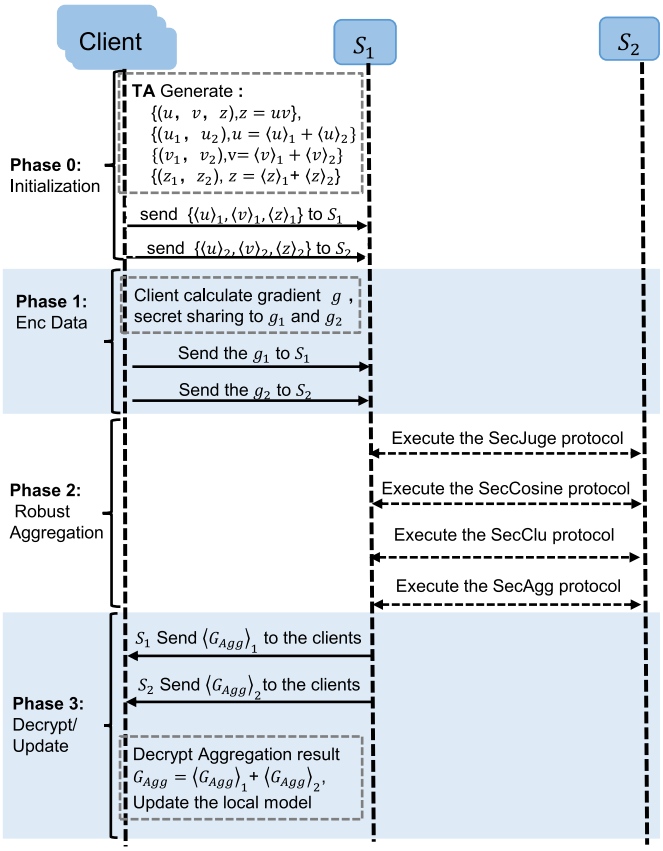


Fig. 4. Overview of PDFL.

Phase 0 *(Initialization)*: The *TA* initializes the entire system and distributes certain parameters to the clients. The clients employ these parameters to encrypt the data.

Phase 1 *(Clients Train Model)*: Clients train the local model according to their own data set, split the gradients into two secret shares, and distribute one share to each server.

Phase 2 *(Servers Robust Aggregation)*: Servers S_1 and S_2 interactively execute four protocols:the SecJudge, the SecCosine, the SecClu and the SecAgg, which ensure a robust aggregation of the gradients from all clients.

Phase 3 *(Update Local Model)*: Clients receive the partially aggregated gradients from each server. They then combine these partial results to obtain the final aggregated gradient and update their local models accordingly.

C. Construction of PDFL

In this phase, we design four building blocks as the basic security protocols for PDFL: SecJudge for gradient normalization, SecCosine for computing the cosine similarity between gradients, SecClu for mitigating poisoning attacks, and SecAgg for secure server-side aggregation. As shown in *Fig. 5*, the secret sharing scheme is employed multiple times across the four protocols to enhance privacy.

1) The SecJudge Protocol: Alg. 1 shows the specific steps of SecJudge. Upon receiving the split gradient $\langle g_i\rangle_1 = \{\langle g_{i_1}\rangle_1, \langle g_{i_2}\rangle_1, \ldots, \langle g_{i_m}\rangle_1\}$ and $\langle g_i\rangle_2 = \{\langle g_{i_1}\rangle_2, \langle g_{i_2}\rangle_2, \ldots, \langle g_{i_m}\rangle_2\}$ from the client i, S_1 and S_2 interactively determine whether the local gradients are normalized or not. The SecJudge protocol involves the multiplication of secret sharing $Mul(\cdot)$. Server S_1 interacts Frist with S_2 to obtain

$$g_{i_{\text{sum}}} = g_{i_1}^2 + g_{i_2}^2 + \ldots + g_{i_m}^2.$$

Server S_1 judges wheather $g_{i_{\text{sum}}} = 1$, if it is true, S_1 accepts the gradient g_i. Otherwise, it will abandon the gradient g_i and does not include it in the aggregation operation.

2) The SecCosine Protocol: Let $g_i = \{g_{i_1}, g_{i_2}, \ldots, g_{i_m}\}$ be the client i's local gradient, and $g_j = \{g_{j_1}, g_{j_2}, \ldots, g_{j_m}\}$ be the client j's, the cosine similarity value $Cosine(i, j), (i, j \in [1, n])$ of the two gradients g_i and g_j is defined as:

$$Cosine(i, j) = \frac{< g_i, g_j >}{\|g_i\|\|g_j\|} =< g_i, g_j >,$$
$$s.t., \|g_i\| = \|g_j\| = 1. \tag{17}$$

If the two gradients g_i and g_j are normalized vectors, their cosine similarity will be the inner product of two gradients.

Alg. 2 shows the specific steps of SecCosine, the local gradient of client i (resp. client j) g_i (resp. g_j) is split to $\langle g_i\rangle_1$ and $\langle g_i\rangle_2$ (resp. $\langle g_j\rangle_1$ and $\langle g_j\rangle_2$). To implement the cosine similarity among the secret sharing values $\langle g_i\rangle_1$, $\langle g_i\rangle_2$, $\langle g_j\rangle_1$, $\langle g_j\rangle_2$, Servers S_1 and S_2 involve secret sharing multiplication $Mul(\cdot)$, to calculate securely $\{\langle cos_{ij}\rangle_1$, $\langle cos_{ij}\rangle_2\} \leftarrow Mul\{\langle g_i\rangle_1, \langle g_i\rangle_2, \langle g_j\rangle_1, \langle g_j\rangle_2\}$. The cosine similarity value among gradients g_j and gradients g_i will be $cos_{ij} = \langle cos_{ij}\rangle_1 + \langle cos_{ij}\rangle_2$.

3) The SecClu Protocol: For each client i, Servers S_1 and S_2 collaboratively determine clusters by securely computing the cosine similarity of their local gradients. Clients whose gradients exhibit high cosine similarity are grouped into the same cluster.

Alg. 3 outlines the specific steps of SecClu protocol. To perform clustering on encrypted similarity values, the cosine similarity is blinded via multiplication with a random number r, which involves the secret sharing multiplication $Mul(\cdot)$ between the two servers S_1 and S_2. The blinded similarity matrix is then sent to server S_2, which executes the clustering algorithm using these secured values. Finally, this algorithm partitions the client set C into two distinct clusters c_1 and c_2.

$$C = \{\arg \max_{c \in \{c_1, c_2\}} |c|\}. \tag{18}$$

The cluster with the majority of clients C_{Benign} is kept as benign clients.

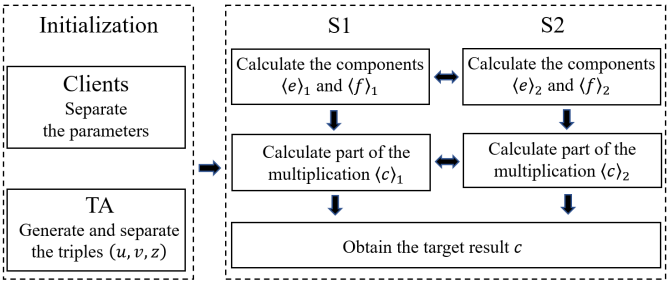


Fig. 5. Application of Secret Sharing in Protocols.

Authorized licensed use limited to: Anhui University. Downloaded on October 18,2026 at 03:38:23 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 The Secure Judge Protocol (SecJudge)

Input: In S_1: local gradient $\langle g_i\rangle_1$, and the secret sharing parts $\langle U\rangle_1$, $\langle V\rangle_1$ and $\langle Z\rangle_1$;
In S_2: local gradient $\langle g_i\rangle_2$, $\langle U\rangle_2$, $\langle V\rangle_2$ and $\langle Z\rangle_2$.
Output: *Accept* g_i or *Abort* g_i.
// Both severs calculate separately the components of the secret sharing according to formula (12)
In S_1:
$\langle e_i\rangle_1 = \langle g_i\rangle_1 - \langle U\rangle_1$
$\langle f_i\rangle_1 = \langle g_i\rangle_1 - \langle V\rangle_1$
Server S_1 sends $\langle e_i\rangle_1$ and $\langle f_i\rangle_1$ to Server S_2
In S_2:
$\langle e_i\rangle_2 = \langle g_i\rangle_2 - \langle U\rangle_2$
$\langle f_i\rangle_2 = \langle g_i\rangle_2 - \langle V\rangle_2$
Server S_2 sends $\langle e_i\rangle_2$ and $\langle f_i\rangle_2$ to Server S_1
In S_1 and S_2:
$e_i = \langle e_i\rangle_1 + \langle e_i\rangle_2$
$f_i = \langle f_i\rangle_1 + \langle f_i\rangle_2$
// Reconstruct $\langle g\rangle$ according to (13)--(16)
In S_1:
for $k = 1$ to m **do**
  $\langle g_{i_k}\rangle_1 = e_{i_k} f_{i_k} + \langle u_k\rangle_1 f_{i_k} + \langle v_k\rangle_1 e_{i_k} + \langle z_k\rangle_1$
end for
$\langle g_{i_{\text{sum}}}\rangle_1 = \sum_{k=1}^{m} \langle g_{i_k}\rangle_1$
In S_2:
for $k = 1$ to m **do**
  $\langle g_{i_k}\rangle_2 = \langle u_k\rangle_2 f_{i_k} + \langle v_k\rangle_2 e_{i_k} + \langle z_k\rangle_2$
end for
$\langle g_{i_{\text{sum}}}\rangle_2 = \sum_{i=1}^{n} \langle g_{i_k}\rangle_2$
In S_1 and S_2:
$g_{i_{\text{sum}}} = \langle g_{i_{\text{sum}}}\rangle_1 + \langle g_{i_{\text{sum}}}\rangle_2$
// Verify if the local gradient is normalized
if $g_{i_{\text{sum}}} == 1$ **then**
  Accept g_i
else
  Abort g_i
end if
Return: *Accept* g_i or *Abort* g_i

Algorithm 2 The Secure Cosine Protocol (SecCosine)

Input: S_1: local gradient $\langle g_i\rangle_1$, $\langle g_j\rangle_1$, $\langle U\rangle_1$, $\langle V\rangle_1$ and $\langle Z\rangle_1$;
In S_2: local gradient $\langle g_i\rangle_2$, $\langle g_j\rangle_2$, $\langle U\rangle_2$, $\langle V\rangle_2$ and $\langle Z\rangle_2$.
Output: $\langle \eta_{ij}\rangle_1$ and $\langle \eta_{ij}\rangle_2$.
// Both severs calculate separately the components of the secret sharing according to formula (12)
In S_1:
$\langle e_i\rangle_1 = \langle g_i\rangle_1 - \langle U\rangle_1$
$\langle f_j\rangle_1 = \langle g_j\rangle_1 - \langle V\rangle_1$
Server S_1 sends $\langle e_i\rangle_1$ and $\langle f_j\rangle_1$ to server S_2
In S_2:
$\langle e_i\rangle_2 = \langle g_i\rangle_2 - \langle U\rangle_2$
$\langle f_j\rangle_2 = \langle g_j\rangle_2 - \langle V\rangle_2$
Server S_2 sends $\langle e_i\rangle_2$ and $\langle f_j\rangle_2$ to server S_1
// Both severs reconstruct the components according to formula (13)
In S_1 and S_2:
$e_i = \langle e_i\rangle_1 + \langle e_i\rangle_2$
$f_j = \langle f_j\rangle_1 + \langle f_j\rangle_2$
// Both severs construct separately the cosine similarity part according to (14) and (15)
In S_1:
for $k = 1$ to m **do**
  $\langle D_{ij_k}\rangle_1 = e_{i_k} f_{j_k} + \langle u_k\rangle_1 f_{j_k} + \langle v_k\rangle_1 e_{i_k} + \langle z_k\rangle_1$
end for
$\langle cos_{ij}\rangle_1 = \sum_{k=1}^{k=m} \langle D_{ij_k}\rangle_1$
$\langle \eta_{ij}\rangle_1 = \langle cos_{ij}\rangle_1$
In S_2:
for $k = 1$ to m **do**
  $\langle D_{ij_k}\rangle_2 = \langle u_k\rangle_2 f_{j_k} + \langle v_k\rangle_2 e_{i_k} + \langle z_k\rangle_2$
end for
$\langle cos_{ij}\rangle_2 = \sum_{k=1}^{k=m} \langle D_{ij_k}\rangle_2$
$\langle \eta_{ij}\rangle_2 = \langle cos_{ij}\rangle_2$
Return: $\langle \eta_{ij}\rangle_1$ and $\langle \eta_{ij}\rangle_2$

4) The SecAgg Protocol: To defend the global model against local poisoning attacks, PDFL employs a secure aggregation mechanism based on the benign clients' gradients and cosine similarity. The specifics of this SecAgg protocol are presented in *Alg. 4*.

Server S_1 chooses the cosine similarity $\langle \eta_{ij}\rangle_1$, where $i, j \in C_{\text{Benign}}$, and calculates the average value $\langle \eta_i\rangle_1$. Similarly for Server S_2 who obtain another average value $\langle \eta_i\rangle_2$. Denote $\eta_i = \langle \eta_i\rangle_1 + \langle \eta_i\rangle_2$, which is in fact the weight of gradients,

$$\eta_i = \sum_{j \in C_{\text{Benign}}} \eta_{ij} / |C_{\text{Benign}}|. \tag{19}$$

To implement the secret sharing multiplication of $Mul\{\langle \eta_i\rangle_1, \langle \eta_i\rangle_2, \langle g_i\rangle_1, \langle g_i\rangle_2\}$. Servers S_1 and S_2 will collaboratively calculate $\langle e_i\rangle = \langle g_i\rangle_1 - \langle U\rangle_1$ and $\langle f_i\rangle = \langle g_i\rangle_1 - \langle V\rangle_1$, and $\eta = \langle \eta\rangle_1 + \langle \eta\rangle_2$.

$$\eta = \sum_{i \in C_{\text{Benign}}} \eta_i. \tag{20}$$

Servers S_1 and S_2 calculate respectively the sharing of aggregation result $\langle Agg_{C_{\text{Benign}}}\rangle_1$ and $\langle Agg_{C_{\text{Benign}}}\rangle_2$, and send the result to all clients. Upon receiving these secret sharings, each client will reconstruct the global model aggregation result $Agg_{C_{\text{Benign}}}$,

$$Agg_{C_{\text{Benign}}} = \langle Agg_{C_{\text{Benign}}}\rangle_1 + \langle Agg_{C_{\text{Benign}}}\rangle_2. \tag{21}$$

D. Implementation of PDFL

1) Initialization Phase: During the initialization phase, the TA generates the secret sharing parameters, and distributes them to the two servers. The TA then goes offline.

(i) *Beaver Triple Generation*: TA chooses $2m$ random numbers $\{u_1, u_2, \ldots, u_m\}$ and $\{v_1, v_2, \ldots, v_m\}$ from $\mathbb{Z}_{2^l}$, and calculates $z_i = u_i v_i$. $U = \{u_1, u_2, \ldots, u_m\}$, $V = \{v_1, v_2, \ldots, v_m\}$ and $Z = \{z_1, z_2, \ldots, z_m\}$.

Authorized licensed use limited to: Anhui University. Downloaded on October 18,2026 at 03:38:23 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3 The Secure Clustering Protocol (SecClu)

Input: Initial clusters $C = \{\{1, 2, \ldots, n\}\}$
In S_1: $\{\langle \eta_{ij} \rangle_1, \forall i, j \in [1, n]\}$, $\langle u \rangle_1$, $\langle v \rangle_1$, $\langle z \rangle_1$, and random number $\langle r \rangle_1$;
In S_2: $\{\langle \eta_{ij} \rangle_2, \forall i, j \in [1, n]\}$, $\langle u \rangle_2$, $\langle v \rangle_2$, $\langle z \rangle_2$, and random number $\langle r \rangle_2$
Output: Benign client's ID set C_{Benign}.
// Both severs calculate separately the components of the secret sharing according to formula (12)
In S_1:
$\langle e_{ij} \rangle_1 = \langle \eta_{ij} \rangle_1 - \langle u \rangle_1$, $\forall i, j \in [1, n]$
$\langle f \rangle_1 = \langle r \rangle_1 - \langle v \rangle_1$
Server S_1 sends $\{\langle e_{ij} \rangle_1, \forall i, j \in [1, n]\}$ and $\langle f \rangle_1$ to server S_2
In S_2:
$\langle e_{ij} \rangle_2 = \langle \eta_{ij} \rangle_2 - \langle u \rangle_2$, $\forall i, j \in [1, n]$
$\langle f \rangle_2 = \langle r \rangle_2 - \langle v \rangle_2$
Server S_2 sends $\{\langle e_{ij} \rangle_2, \forall i, j \in [1, n]\}$ and $\langle f \rangle_2$ to server S_1
// Both severs reconstruct the components according to formula (13)
In S_1 and S_2:
$e_{ij} = \langle e_{ij} \rangle_1 + \langle e_{ij} \rangle_2$, $\forall i, j \in [1, n]$
$f = \langle f \rangle_1 + \langle f \rangle_2$
// Both severs construct separately the similarity part according to (14) and (15) and then multiplicate
In S_1:
$\langle D_{ij} \rangle_1 = e_{ij} f + \langle u \rangle_1 f + \langle v \rangle_1 e_{ij} + \langle z \rangle_1$, $\forall i, j \in [1, n]$ Server S_1 sends $\{\langle D_{ij} \rangle_1, \forall i, j \in [1, n]\}$ to Server S_2.
In S_2
$\langle D_{ij} \rangle_2 = \langle u \rangle_2 f + \langle v \rangle_2 e_{ij} + \langle z \rangle_2$, $\forall i, j \in [1, n]$
$D_{ij} = \langle D_{ij} \rangle_1 + \langle D_{ij} \rangle_2$, $\forall i, j \in [1, n]$
// Partition the client set into two distinct clusters and take the majority as benign.
for $c \in C$ **do**
$c_1, c_2 \leftarrow \arg\min_{c_1 \cup c_2 = c} (\max_{i \in c_1, j \in c_2} (D_{ij}))$
$D_{\text{across}} \leftarrow \max_{i \in c_1, j \in c_2} D_{ij}$
if $D_{\text{across}} < a_{\text{across}}^{\text{threshold}}$ **then**
$C \leftarrow C \setminus c \cup c_1 \cup c_2$
$C \leftarrow \{\arg\max_{c \in \{c_1, c_2\}} |c|\}$
end if
end for
$C_{\text{Benign}} = C$
Return: C_{Benign}

(ii) *Secret Sharing*: TA splits the paraments U, V, Z to two pieces, and calculates $\{\langle U \rangle_1, \langle U \rangle_2\}$, $\{\langle V \rangle_1, \langle V \rangle_2\}$ and $\{\langle Z \rangle_1, \langle Z \rangle_2\}$.

$$\begin{cases} U = \langle U \rangle_1 + \langle U \rangle_2 \\ V = \langle V \rangle_1 + \langle V \rangle_2 \\ Z = \langle Z \rangle_1 + \langle Z \rangle_2. \end{cases} \tag{22}$$

The TA sends $\{\langle U \rangle_1, \langle V \rangle_1, \langle Z \rangle_1\}$ to Server S_1, and $\{\langle U \rangle_2, \langle V \rangle_2, \langle Z \rangle_2\}$ to Server S_2.

Algorithm 4 The Secure Aggregation Protocol (SecAgg)

Input: Benign client ID C_{Benign}
In S_1: $\langle g_i \rangle_1$, $\{\langle \eta_{ij} \rangle_1, \forall i, j \in [1, n]\}$, $\langle U \rangle_1$, $\langle V \rangle_1$ and $\langle Z \rangle_1$
In S_2: $\langle g_i \rangle_2$, $\{\langle \eta_{ij} \rangle_2, \forall i, j \in [1, n]\}$, $\langle U \rangle_2$, $\langle V \rangle_2$ and $\langle Z \rangle_2$
Output: $\langle Agg_{C_{\text{Benign}}} \rangle_1$ and $\langle Agg_{C_{\text{Benign}}} \rangle_2$
// Both severs calculate separately the components of the secret sharing and exchange the results
In S_1.
for $i \in C_{\text{Benign}}$ **do**
$\langle \eta_i \rangle_1 = \sum_{j \in C_{\text{Benign}}} \langle \eta_{ij} \rangle_1 / |C_{\text{Benign}}|$
$\langle e_i \rangle_1 = \langle g_i \rangle_1 - \langle U \rangle_1$
$\langle f_i \rangle_1 = \langle \eta_i \rangle_1 - \langle V \rangle_1$
end for
$\langle \eta \rangle_1 = \sum_{i \in C_{\text{Benign}}} \langle \eta_i \rangle_1$
S_1 sends $\langle \eta \rangle_1$, $\{\langle e_i \rangle_1, \forall i \in C_{\text{Benign}}\}$ and $\{\langle f_i \rangle_1, \forall i \in C_{\text{Benign}}\}$ to S_2
In S_2
for $i \in C_{\text{Benign}}$ **do**
$\langle \eta_i \rangle_2 = \sum_{j \in C_{\text{Benign}}} \langle \eta_{ij} \rangle_2 / |C_{\text{Benign}}|$
$\langle e_i \rangle_2 = \langle g_i \rangle_2 - \langle U \rangle_2$
$\langle f_i \rangle_2 = \langle \eta_i \rangle_2 - \langle V \rangle_2$
end for
$\langle \eta \rangle_2 = \sum_{i \in C_{\text{Benign}}} \langle \eta_i \rangle_2$
S_2 sends $\langle \eta \rangle_2$, $\{\langle e_i \rangle_2, \forall i \in C_{\text{Benign}}\}$ and $\{\langle f_i \rangle_2, \forall i \in C_{\text{Benign}}\}$ to S_1
// Both severs reconstruct the multiplication components
In S_1 and S_2:
for $i \in C_{\text{Benign}}$ **do**
$e_i = \langle e_i \rangle_1 + \langle e_i \rangle_2$
$f_i = \langle f_i \rangle_1 + \langle f_i \rangle_2$
end for
$\eta = \langle \eta \rangle_1 + \langle \eta \rangle_2$
// Both severs construct separately the multiplication component
In S_1
for $i \in C_{\text{Benign}}$ **do**
$\langle D_i \rangle_1 = e_i f_i + \langle U \rangle_1 f_i + \langle V \rangle_1 e_i + \langle Z \rangle_1$
end for
$\langle Agg_{C_{\text{Benign}}} \rangle_1 = \sum_{i \in C_{\text{Benign}}} \langle D_i \rangle_1 / \eta$
In S_2
for $i \in C_{\text{Benign}}$ **do**
$\langle D_i \rangle_2 = \langle U \rangle_2 f_i + \langle V \rangle_2 e_i + \langle Z \rangle_2$,
end for
// Aggregate
$\langle Agg_{C_{\text{Benign}}} \rangle_2 = \sum_{i \in C_{\text{Benign}}} \langle D_i \rangle_2 / \eta$.
Return: $\langle Agg_{C_{\text{Benign}}} \rangle_1$ and $\langle Agg_{C_{\text{Benign}}} \rangle_2$.

2) Privacy Preserving of PDFL: Alg. 5 shows the implementation of PDFL, which is composed of three parts: (*i*) Client Encrypt (*ii*) Server Robust Aggregation (*iii*) Client Decrypt and Update.

(i) *Client Encrypt*: During the t-th iteration, client i locally trains the model and shares the gradients:

$$L_f\left(D^{*t}, \Theta_t\right) = \frac{1}{|D^{*t}|} \sum_{\langle x_j, y_j \rangle \in D^{*t}} \left(y_j - f\left(x_j, \Theta_t\right)\right)^2, \tag{23}$$

$$g_t = \nabla L_f\left(D^{*t}, \Theta_t\right). \tag{24}$$

Client i splits the gradient g_i^t into $\langle g_i^t \rangle_1$ and $\langle g_i^t \rangle_2$:

$$\begin{cases} \langle g_i^t \rangle_1 = \{\langle g_{i_1}^t \rangle_1, \langle g_{i_2}^t \rangle_1, \ldots, \langle g_{i_m}^t \rangle_1\} \\ \langle g_i^t \rangle_2 = \{\langle g_{i_1}^t \rangle_2, \langle g_{i_2}^t \rangle_2, \ldots, \langle g_{i_m}^t \rangle_2\}. \end{cases} \tag{25}$$

(ii) *Server Robust Aggregation*: Upon receiving the encrypted gradient from the clients, S_1 and S_2

Authorized licensed use limited to: Anhui University. Downloaded on October 18,2026 at 03:38:23 UTC from IEEE Xplore. Restrictions apply.

Algorithm 5 Implementation of PDFL

Input: Dataset $D^{*t} = \{(x_i, y_i), i = 1, 2, \ldots, T\}$, model paraments Θ_t, learning rate α

Output: Model paraments Θ_{t+1}

In *Client i*: `// Calculate local parameters`

$L_f\left(D^{*t}, \Theta_t\right) = \frac{1}{|D^{*t}|}\sum_{\langle x_i, y_i\rangle \in D^{*t}}(y_i - f(x_i, \Theta_t))^2$

$g_t = \nabla L_f\left(D^{*t}, \Theta_t\right)$

$\langle g_i^t\rangle_1 = \langle g_{i_1}^t\rangle_1, \langle g_{i_2}^t\rangle_1, \ldots, \langle g_{i_m}^t\rangle_1$

$\langle g_i^t\rangle_2 = \langle g_{i_1}^t\rangle_2, \langle g_{i_2}^t\rangle_2, \ldots, \langle g_{i_m}^t\rangle_2$

In S_1 and S_2: `// Aggregate securely`

for $i = 1$ to n **do**

  $g_{i_{\text{sum}}}^t \leftarrow SecJudge\{\langle g_i^t\rangle_1, \langle g_i^t\rangle_2\}$

  if $g_{i_{\text{sum}}}^t \neq 1$ **then**

    $C \leftarrow C/Client\ i$

  end if

end for

for $i \in C$ **do**

  for $j \in C$ **do**

    $\{\langle \eta_{ij}^t\rangle_1, \langle \eta_{ij}^t\rangle_2\} \leftarrow$

    $SecCosine\{\langle g_i^t\rangle_1, \langle g_i^t\rangle_2, \langle g_j^t\rangle_1, \langle g_j^t\rangle_2\}$

  end for

end for

$C_{\text{Benign}} \leftarrow SecClu\{\langle \eta_{ij}^t\rangle_1, \langle \eta_{ij}^t\rangle_2\}$

$\{\langle Agg_{C_{\text{Benign}}}^t\rangle_1, \langle Agg_{C_{\text{Benign}}}^t\rangle_2\} \leftarrow$

$SecAgg\{\langle g_i^t\rangle_1, \langle g_i^t\rangle_2, \langle \eta_{ij}^t\rangle_1, \langle \eta_{ij}^t\rangle_2, C_{\text{Benign}}\}$

In *Client i*: `// Update the parameters`

$Agg_{C_{\text{Benign}}}^t = \langle Agg_{C_{\text{Benign}}}^t\rangle_1 + \langle Agg_{C_{\text{Benign}}}^t\rangle_2$

$\Theta_{t+1} = \Theta_t - \alpha Agg_{C_{\text{Benign}}}^t$

implement $g_{i_{\text{sum}}}^t \leftarrow SecJudge\{\langle g_i^t\rangle_1, \langle g_i^t\rangle_2\}$, if $g_{i_{\text{sum}}}^t == 1$, the servers S_1 and S_2 calculate on turn

$$\{\langle \eta_{ij}^t\rangle_1, \langle \eta_{ij}^t\rangle_2\} \leftarrow SecCosine\{\langle g_i^t\rangle_1, \langle g_i^t\rangle_2, \langle g_j^t\rangle_1, \langle g_j^t\rangle_2\}, \tag{26}$$

the benign clients cluster,

$$C_{\text{Benign}} \leftarrow SecClu\{\langle \eta_{ij}^t\rangle_1, \langle \eta_{ij}^t\rangle_2\}, \tag{27}$$

and then robustly aggregate the gradient

$$\{\langle Agg_{C_{\text{Benign}}}^t\rangle_1, \langle Agg_{C_{\text{Benign}}}^t\rangle_2\} \leftarrow SecAgg\{\langle g_i^t\rangle_1, \langle g_i^t\rangle_2, \langle \eta_{ij}^t\rangle_1, \langle \eta_{ij}^t\rangle_2, C_{\text{Benign}}\}. \tag{28}$$

Finally, the aggregated results are distributed to all clients.

(iii) *Client Decrypt and Update*: Client employs the secret sharing of global model aggregation results $\langle Agg_{C_{\text{Benign}}}^t\rangle_1$ and $\langle Agg_{C_{\text{Benign}}}^t\rangle_2$ to aggregate and update the local model:

$$Agg_{C_{\text{Benign}}}^t = \langle Agg_{C_{\text{Benign}}}^t\rangle_1 + \langle Agg_{C_{\text{Benign}}}^t\rangle_2, \tag{29}$$

$$\Theta_{t+1} = \Theta_t - \alpha Agg_{C_{\text{Benign}}}^t. \tag{30}$$

V. Effectiveness Analysis for PDFL

A. Security Analysis

The threat model is defined in Section II-B, where we supposed that the severs are honest-but-curious. We prove in this section that PDFL achieves Sim-based security under the *Definition 3*:

Definition 3 (Simulation-Based Security)): Suppose there exists an attacker $\mathcal{A}$ interacting with a challenger in the real world to execute protocol $\prod$. If there is a polynomial-time simulator $\mathcal{S}$ such that attacker $\mathcal{A}$ interacts with simulator $\mathcal{S}$ in the ideal world to complete protocol $\prod$, and the view obtained by the attacker from the ideal world is computationally indistinguishable from the view obtained in the real world, then it can be proven that this protocol is secure.

$$Real_{\mathcal{A}}^{\prod}(\epsilon, g) \overset{c}{\equiv} Ideal_{\mathcal{A},\mathcal{S}}^{\prod}(\epsilon', g'), \tag{31}$$

where $\overset{c}{\equiv}$ denotes computational indistinguishability. $Real_{\mathcal{A}}^{\prod}$ represents the view obtained by the attacker from the real world, $Ideal_{\mathcal{A},\mathcal{S}}^{\prod}$ represents the view obtained by the attacker from the ideal world, ϵ is the security parameter, and g is the input of the $\prod$ representing the protocol.

Theorem 1: In the semi-honest model, the SecJudge protocol is secure which guarantees data security and privacy preservation.

Proof: The real world view of S_1 can be represented as $Real_{\mathcal{A}}^{\prod}\{\langle g_i\rangle_1, \langle e_i\rangle_1, \langle f_i\rangle_1, \langle g_{i_{\text{sum}}}\rangle_1\}$, where $\langle g_i\rangle_1 = g_i - r$, r is a random number of Z_N. The view of S_1 in an ideal world could be represented as $Ideal_{\mathcal{A},\mathcal{S}}^{\prod}\{\langle g_i'\rangle_1, \langle e_i'\rangle_1, \langle f_i'\rangle_1, \langle g_{i_{\text{sum}}}'\rangle_1\}$, where $\langle g_i'\rangle_1, \langle e_i'\rangle_1, \langle f_i'\rangle_1$ and $\langle g_{i_{\text{sum}}}'\rangle_1$ are random numbers of Z_N. Therefore, $\langle g_i\rangle_1, \langle e_i\rangle_1, \langle f_i\rangle_1, \langle g_{i_{\text{sum}}}\rangle_1$ and $\langle g_i'\rangle_1, \langle e_i'\rangle_1, \langle f_i'\rangle_1, \langle g_{i_{\text{sum}}}'\rangle_1$ are computationally indistinguishable, thus:

$$Real_{\mathcal{A}}^{\prod}\{\langle g_i\rangle_1, \langle e_i\rangle_1, \langle f_i\rangle_1, \langle g_{i_{\text{sum}}}\rangle_1\} \overset{c}{\equiv} Ideal_{\mathcal{A},\mathcal{S}}^{\prod}\{\langle g_i'\rangle_1, \langle e_i'\rangle_1, \langle f_i'\rangle_1, \langle g_{i_{\text{sum}}}'\rangle_1\}. \tag{32}$$

Subsequentely, Server S_1 can not learn anything about g_i, nor can S_2.

Theorem 2: In the semi-honest model, the SecCosine protocol is secure, which guarantees data security and privacy preservation.

Proof: The real world view of S_1 can be represented as $Real_{\mathcal{A}}^{\prod}\{\langle g_i\rangle_1, \langle e_i\rangle_1, \langle f_i\rangle_1, \langle D_{ij}\rangle_1, \langle cos_{ij}\rangle_1, \langle \eta_{ij}\rangle_1\}$, where $\langle g_i\rangle_1 = g_i - r$ is a random numbers of Z_N. The view of S_1 in an ideal world could be represented as $Ideal_{\mathcal{A},\mathcal{S}}^{\prod}\{\langle g_i'\rangle_1, \langle e_i'\rangle_1, \langle f_i'\rangle_1, \langle D_{ij}'\rangle_1, \langle cos_{ij}'\rangle_1, \langle \eta_{ij}'\rangle_1\}$, where $\langle g_i'\rangle_1, \langle e_i'\rangle_1, \langle f_i'\rangle_1, \langle D_{ij}'\rangle_1, \langle cos_{ij}'\rangle_1$ and $\langle \eta_{ij}'\rangle_1$ are random numbers of Z_N. $\langle g_i\rangle_1, \langle e_i\rangle_1, \langle f_i\rangle_1, \langle D_{ij}\rangle_1, \langle cos_{ij}\rangle_1, \langle \eta_{ij}\rangle_1$ and $\langle g_i'\rangle_1, \langle e_i'\rangle_1, \langle f_i'\rangle_1, \langle D_{ij}'\rangle_1, \langle cos_{ij}'\rangle_1, \langle \eta_{ij}'\rangle_1$ are computationally indistinguishable, thus:

$$Real_{\mathcal{A}}^{\prod}\{\langle g_i\rangle_1, \langle e_i\rangle_1, \langle f_i\rangle_1, \langle D_{ij}\rangle_1, \langle cos_{ij}\rangle_1, \langle \eta_{ij}\rangle_1\} \overset{c}{\equiv} Ideal_{\mathcal{A},\mathcal{S}}^{\prod}\{\langle g_i'\rangle_1, \langle e_i'\rangle_1, \langle f_i'\rangle_1, \langle D_{ij}'\rangle, \langle cos_{ij}'\rangle_1, \langle \eta_{ij}'\rangle_1\}. \tag{33}$$

Server S_1 can not learn anything about g_i, nor can S_2.

Theorem 3: In the semi-honest model, the SecClu protocol is secure which can ensure the data security and privacy.

Proof: The real world view of S_2 can be represented as $Real_{\mathcal{A}}^{\prod}\{\langle \eta_{ij}\rangle_1, \langle e_{ij}\rangle_1, \langle f\rangle_1, \langle D_{ij}\rangle_1\}$, where $\langle \eta_{ij}\rangle_1 = \eta_{ij} - r$, r is a

Authorized licensed use limited to: Anhui University. Downloaded on October 18,2026 at 03:38:23 UTC from IEEE Xplore. Restrictions apply.

random numbers of Z_N. The view of S_1 in an ideal world could be represented as $Ideal^{\prod}_{\mathcal{A},S}\{\langle\eta'_{ij}\rangle_1, \langle e'_{ij}\rangle_1, \langle f'\rangle_1, \langle D'_{ij}\rangle_1\}$, where $\langle\eta'_{ij}\rangle_1, \langle e'_{ij}\rangle_1, \langle f'\rangle_1$ and $\langle D'_{ij}\rangle_1$ are random numbers of Z_N. $\langle\eta_{ij}\rangle_1, \langle e_{ij}\rangle_1, \langle f\rangle_1, \langle D_{ij}\rangle_1$ and $\langle\eta'_{ij}\rangle_1, \langle e'_{ij}\rangle_1, \langle f'\rangle_1, \langle D'_{ij}\rangle_1$ are computationally indistinguishable, thus:

$$Real^{\prod}_{\mathcal{A}}\{\langle\eta_{ij}\rangle_1, \langle e_{ij}\rangle_1, \langle f\rangle_1, \langle D_{ij}\rangle_1\} \overset{c}{\equiv} Ideal^{\prod}_{\mathcal{A},S}\{\langle\eta'_{ij}\rangle_1, \langle e'_{ij}\rangle_1, \langle f'\rangle_1, \langle D'_{ij}\rangle_1\}. \tag{34}$$

Server S_1 can not learn anything about η_{ij}, nor S_2.

Theorem 4: In the semi-honest model, the SecAgg protocol is secure which can ensure the data security and privacy.

Proof: The real world view of S_2 can be represented as $Real^{\prod}_{\mathcal{A}}\{\langle g_i\rangle_1, \langle\eta_{ij}\rangle_1, \langle e_i\rangle_1, \langle f_i\rangle_1, \langle Agg_{C_{\text{Benign}}}\rangle_1\}$, where $\langle g_i\rangle_1 = g_i - r$, r is a random numbers of Z_N. The view of S_1 in an ideal world could be represented as $Ideal^{\prod}_{\mathcal{A},S}\{\langle g'_i\rangle_1, \langle\eta_{ij}\rangle_1, \langle e'_i\rangle_1, \langle f'_i\rangle_1, \langle Agg'_{C_{\text{Benign}}}\rangle_1\}$, where $\langle g'_i\rangle_1, \langle\eta'_{ij}\rangle_1, \langle e'_i\rangle_1, \langle f'_i\rangle_1$ and $\langle Agg'_{C_{\text{Benign}}}\rangle_1$ are random numbers of Z_N. $\langle g_i\rangle_1, \langle\eta_{ij}\rangle_1, \langle e_i\rangle_1, \langle f_i\rangle_1, \langle Agg_{C_{\text{Benign}}}\rangle_1$ and $\langle g'_i\rangle_1, \langle\eta'_{ij}\rangle_1, \langle e'_i\rangle_1, \langle f'_i\rangle_1, \langle Agg'_{C_{\text{Benign}}}\rangle_1$ are computationally indistinguishable, thus:

$$Real^{\prod}_{\mathcal{A}}\{\langle g_i\rangle_1, \langle\eta_{ij}\rangle_1, \langle e_i\rangle_1, \langle f_i\rangle_1, \langle Agg_{C_{\text{Benign}}}\rangle_1\} \overset{c}{\equiv} Ideal^{\prod}_{\mathcal{A},S}\{\langle g'_i\rangle_1, \langle\eta'_{ij}\rangle_1, \langle e'_i\rangle_1, \langle f'_i\rangle_1, \langle Agg'_{C_{\text{Benign}}}\rangle_1\}. \tag{35}$$

The server S_1 can not learn anything about g_i and η_{ij}, nor can S_2.

Theorem 5: In the semi-honest model, the PDFL scheme is secure which can ensure the data security and privacy.

Proof: We demonstrate the security of this scheme by standard hybrid argumentation method. There are two non-collusive servers S_1 and S_2, denote $S = \{S_1, S_2\}$. Denote ϵ the security parameter, and n the number of clients. Let $Real^{n,\epsilon}_{A,S}$ represent the view of S. Next we prove that there exists a *PPT* simulator *SIM*, such that the view $Ideal^{\prod}_{\mathcal{A},S}$, generated by an attacker $\mathcal{A}$ interacting with the polynomial simulator in the ideal world to simulate protocol $\prod$, is computationally indistinguishable from the view $Real^{n,\epsilon}_{\mathcal{A},S}$.

Hyb1: Initializes a random variable $Real^{n,\epsilon}_{\mathcal{A},S}\{\langle g_i^t\rangle_1, \langle g_i^t\rangle_2\}$, which represents client i's view at the execution of the real protocol.

Hyb2: In this hybrid, we change the input of clients. Each client i splits the random number $g_i^{t'}$ instead of g_i^t, ensuring that $\langle g_i^t\rangle_1$ is computationally indistinguishable from $\langle g_i^{t'}\rangle_1$, and $\langle g_i^t\rangle_2$ is computationally indistinguishable with $\langle g_i^{t'}\rangle_2$.

Hyb3: In this hybrid, we change the input of the SecJudge protocol, replacing $\langle g_i^t\rangle_1$ and $\langle g_i^t\rangle_2$ with $\langle g_i^{t'}\rangle_1$ and $\langle g_i^{t'}\rangle_2$ respectively, we can calculate $g_{i_{\text{sum}}}^{t'} \leftarrow SecJudge\{\langle g_i^{t'}\rangle_1, \langle g_i^{t'}\rangle_2\}$. The security of the SecJudge protocol guarantees that $g_{i_{\text{ssum}}}^{t'}$ is computationally indistinguishable with $g_{i_{\text{sum}}}^t$.

Hyb4: In this hybrid, we change the input of SecCosine protocol. Each client i encrypts the random number $\langle g_i^{t'}\rangle_1, \langle g_i^{t'}\rangle_2, \langle g_j^{t'}\rangle_1$ and $\langle g_j^{t'}\rangle_2$ instead of $\langle g_i^t\rangle_1, \langle g_i^t\rangle_2, \langle g_j^t\rangle_1$ and $\langle g_j^t\rangle_2$, and calculates $\{\langle\eta_{ij}^{t'}\rangle_1, \langle\eta_{ij}^{t'}\rangle_2\} \leftarrow SecCosine\{\langle g_i^{t'}\rangle_1, \langle g_i^{t'}\rangle_2, \langle g_j^{t'}\rangle_1, \langle g_j^{t'}\rangle_2\}$. The security of SecCosine protocol ensures that $\langle\eta_{ij}^t\rangle_1, \langle\eta_{ij}^t\rangle_2$ is computationally indistinguishable from $\langle\eta_{ij}^{t'}\rangle_1, \langle\eta_{ij}^{t'}\rangle_2$.

Hyb5: In this hybrid, we change the input of SecClu protocol. Each client i encrypts the random number $\langle\eta_{ij}^{t'}\rangle_1, \langle\eta_{ij}^{t'}\rangle_2$ instead of $\langle\eta_{ij}^t\rangle_1, \langle\eta_{ij}^t\rangle_2$, and calculates $C'_{\text{Benign}} \leftarrow SecClu\{\langle\eta_{ij}^{t'}\rangle_1, \langle\eta_{ij}^{t'}\rangle_2\}$. The security of SecClu protocol ensures that C_{Benign} is computationally indistinguishable from C'_{Benign}.

Hyb6: In this hybrid, we change the input of the SecAgg protocol, we raplace $\langle g_i^{t'}\rangle_1, \langle g_i^{t'}\rangle_2, \langle\eta_{ij}^{t'}\rangle_1, \langle\eta_{ij}^{t'}\rangle_2$ and C'_{Benign} with $\langle g_i^t\rangle_1, \langle g_i^t\rangle_2, \langle\eta_{ij}^t\rangle_1, \langle\eta_{ij}^t\rangle_2$ and C_{Benign}. we calculate $\{\langle Agg_{C_{\text{Benign}}}^{t'}\rangle_1, \langle Agg_{C_{\text{Benign}}}^{t'}\rangle_2\} \leftarrow SecAgg\{\langle g_i^{t'}\rangle_1, \langle g_i^{t'}\rangle_2, \langle\eta_{ij}^{t'}\rangle_1, \langle\eta_{ij}^{t'}\rangle_2, C'_{\text{Benign}}\}$. The security of the SecAgg protocol guarantees that $\langle Agg_{C_{\text{Benign}}}^{t'}\rangle_1$ and $\langle Agg_{C_{\text{Benign}}}^{t'}\rangle_2$ are computationally indistinguishable from $\langle Agg_{C_{\text{Benign}}}^t\rangle_1$ and $\langle Agg_{C_{\text{Benign}}}^t\rangle_2$.

Based on the security of the SecJudge protocol, the SecCosine protocol, the SecClu protocol and the SecAgg protocol given above, we prove that there exists a *PPT* simulator *SIM* such that the output of $Ideal^{\prod}_{\mathcal{A},S}$ is computationally computationally indistinguishable from the output of $Real^{n,\epsilon}_{\mathcal{A},S}$.

$$Real^{n,\epsilon}_{\mathcal{A},S}\{\langle g_i^t\rangle_1, \langle g_i^t\rangle_2\} \overset{c}{\equiv} Ideal^{\prod}_{\mathcal{A},S}\{\langle g_i^{t'}\rangle_1, \langle g_i^{t'}\rangle_2\}. \tag{36}$$

B. Functionality

As shown in Table II, we present a functional comparison of our scheme PDFL with ShieldFL [12], PROFL [26], APFED [14] in terms of privacy preserving against servers, privacy preserving against clients, robust aggregation, defense against poisoning attacks and aggregation result privacy.

C. Algorithm Complexity Analysis

The algorithmic complexity of our scheme PDFL is analyzed and compared with ShieldFL [12], PROFL [26] and APFED [14] in Table III.

In PDFL, the servers perform the gradient normalization judgment, the similarity calculation, and the gradient clustering. Assuming M clients participate in the federated learning system over N global iterations, the complexity is analyzed as follows. The SecJudge protocol, executed on each client's gradients per iteration, yields a complexity of $O(MN)$. The SecCosine protocol, which computes pairwise similarities among the M clients each round, has a complexity of $O(M^2N)$. Finally, since only one aggregation and one clustering operation are required in the server side per iteration, the complexities of SecClu and SecAgg are both $O(N)$.

Among the approaches presented in Table II and Table III, several algorithms demonstrate exceptional performance in privacy protection. For instance, ShieldFL provides the same level of privacy protection and aggregation robustness as PDFL. Nonetheless, PDFL achieves lower algorithmic complexity comparing to ShieldFL, which is attributed to the secret

Authorized licensed use limited to: Anhui University. Downloaded on October 18,2026 at 03:38:23 UTC from IEEE Xplore. Restrictions apply.

TABLE II
FUNCTION COMPARISON WITH EXISTING WORKS

Scheme \ Function	Privacy Preserving against Servers	Privacy Preserving against Clients	Robust aggregation	Defense against poisoning attacks	Aggregation result privacy
ShieldFL [12]	√	√	√	√	√
PROFL [26]	√	√	×	√	×
APFED [14]	√	√	×	×	×
Our Scheme	√	√	√	√	√

TABLE III
ALGORITHM COMPLEXITY COMPARISON WITH EXISTING WORKS

Scheme \ Operation	SecJudge	SecMed	SecCosine	SecDis	SecSel	SecClu	SecAgg
ShieldFL [12]	$O(N)$	N/A	$O(MN)$	N/A	N/A	N/A	$O(N)$
PROFL [26]	N/A	$O(MN)$	N/A	$O(M^2N)$	$O(M)$	N/A	$O(N)$
APFed [14]	N/A	$O(N)$	$O(M^2N)$	N/A	N/A	$O(N)$	$O(N)$
Our Scheme	$O(N)$	N/A	$O(M^2N)$	N/A	N/A	$O(N)$	$O(N)$

sharing scheme. Moreover, subsequent experiments demonstrate that PDFL maintains better robustness performance when subjected to backdoor and label-flipping attacks.

VI. PERFORMANCE ANALYSIS

In this part, we evaluate the performance of our scheme PDFL from the aspects of efficiency evaluation, model accuracy and robustness evaluation.

A. Experimental Setup

The simulation experiment of this scheme conducts under Windows 10 system, and the computer is configured with i5-8265U CPU 1.60GHz CPU and 8.00GB RAM. The configuration of experiments is as follows: All experiments are programmed in Python.

Datasets: we employ two benchmark dataset MINIST and CIFAR10.

- **MNIST:** consists of 10 classes of handwritten digits, providing 60000 training samples and 10000 testing samples. Each sample is a 28×28 pixel grayscale image. We employ a LeNet model for training, which consists of 3 convolution layers, 2 pooling layers, 1 fully connected layer, and 1 output layer. The convolution kernel of all the convolution layers is 5×5, the step size is 1, the pooling method is average pooling, and the activation function is sigmoid.
- **CIFAR10:** consist of 10 classes of color images, providing 50000 training samples and 10000 testing samples. Each sample is a 32×32 pixel color image. We use a ResNet for training, comprising three basic Blocks and one linear layer.
- **FashionMNIST**:consists of 10 classes of clothing and accessories, providing 60,000 training samples and 10,000 testing samples. Each sample is a 28×28 pixel grayscale image. We employ an CNN model for training. The CNN includes 2 convolution layers (with 5×5 kernels, stride = 1), 2 pooling layers, 2 fully connected layers, and 1 output layer. The activation function is Sigmoid.

State-of-the-art: we compare our PDFL with two state-of-the-art robustness FL, ShieldFL [12] and PROFL [26].

- **ShieldFL:** designed a privacy-preserving defense strategy using two-trapdoor homomorphic encryption and cosine similarity, which can resist encrypted model poisoning without compromising privacy in FL system.
- **PROFL:** proposed a privacy-preserving Byzantine-robust FL framework, basing on the two-trapdoor additional homomorphic encryption algorithm, the Euclidean distance is used to identify malicious clients poisoning data.

B. Subprotocol Performance

We evaluate firstly the performance of the SecJudge subprotocol, the SecCosine subprotocol, the SecClu subprotocol and the SecAgg subprotocol. We set the length of data is 100. Following 10 evaluations of the subprotocols, the average time and communication costs of the corresponding subprotocols are shown in table IV.

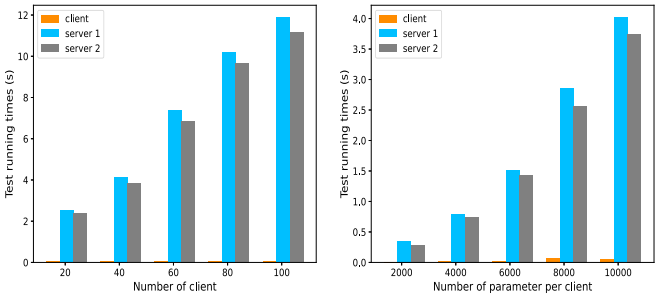
C. Efficiency Evaluation

1) Computation Overhead: In this part, the computational overhead of the PDFL model is evaluated, where we consider mainly the computing time of server S_1, server S_2 and clients. The total number of clients is $m = 100$, the total parament of each client is set as $n = 1000$. *Fig. 5* illustrates the total running time of the PDFL scheme with respect to various number of clients *Fig. 6(a)* and various number of parameters *Fig. 6(b)*. As expected, the computational overhead on the client side does not depend on the total number of clients but increases slightly with the size of local model parameters. In contrast, both factors significantly increase the computational overhead for servers S_1 and S_2. The impact is more pronounced on S_1, as it handles more complex protocol computations.

Authorized licensed use limited to: Anhui University. Downloaded on October 18,2026 at 03:38:23 UTC from IEEE Xplore. Restrictions apply.

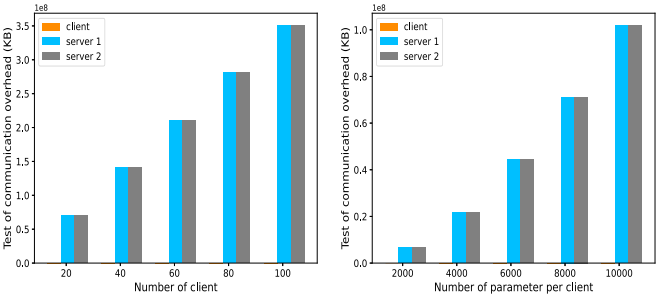
TABLE IV
SUBPROTOCOL PERFORMANCE

Operation \ Overhead	Computational Overhead		Communication Overhead	
	S_1	S_2	S_1	S_2
SecJudge	$0.1089\ ms$	$0.1246\ ms$	$8.08\ KB$	$8.08KB$
SecCosine	$0.10085\ ms$	$0.09918\ ms$	$8.0859\ KB$	$8.0859\ KB$
SecClu	$0.10085\ ms$	$25.98344\ ms$	$8.0859\ KB$	$25.53125\ KB$
SecAgg	$0.1075\ ms$	$0.1020\ ms$	$11.746\ KB$	$11.355\ KB$



(a) (b)

Fig. 6. Computation overhead of PDFL. (a) Computation overhead of PDFL as the number of clients increases. (b) Computation overhead of PDFL as number of paraments increases.



(a) (b)

Fig. 7. Communication overhead of PDFL. (a) Communication overhead of PDFL as the number of clients increases. (b) Communication overhead of PDFL as number of paraments increases.

2) Communication Overhead: *Fig. 7* illustrates the communication overhead between S_1, S_2 and clients. We set the total number of clients as $m = 100$, and the total parament of each client $n = 10000$. We focus on the server-side communication overhead, as the client-side impact is negligible. *Fig. 7(a)* depicts the system's communication cost with varying numbers of clients, while *Fig. 7(b)* examines the cost with different numbers of parameters per client. As the number of clients increases, the communication overhead for both servers S_1 and S_2 rises at a consistent rate, which aligns with the mechanism design.

D. Robustness Evaluation

In this part, we conduct experimental analysis for MNIST, CIFAR10 and FashionMINIST datasets, where 30% (NonIID), 30%(IID), 40% (NonIID) and 40%(IID) poisoning attackers are considered. And the NonIID Dirichlet skewness parameter $q = 0.3$, where Smaller q implies greater label heterogeneity.

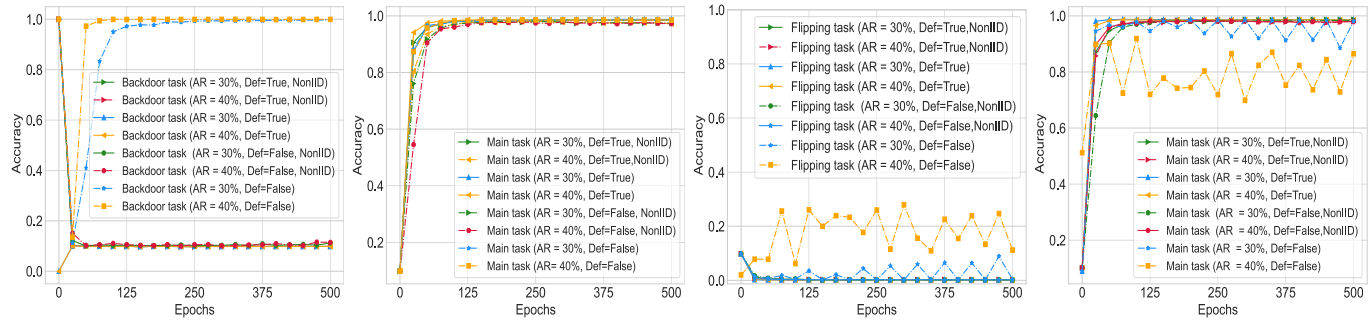
1) MNIST Dataset: *Fig. 6* illustrates the effectiveness of the PDFL defenses against backdoor and flipping attacks on the MINIST dataset.

- **Backdoor Attack:**
  We first evaluate the attack success rate from the adversary's perspective. As shown in *Fig. 8(a)*, the probability of a successful attack is significantly high when no defense mechanism is deployed. This is particularly evident in the case involving 40% malicious clients, where adversaries can quickly and accurately infer private information. In contrast, when the PDFL defense is applied, the attack success rate drops substantially to approximately 10%.
  From the perspective of honest participants, *Fig. 8(b)* present the corresponding model accuracy. The results confirm that the presence of attackers degrades the model's performance, while the implementation of the PDFL defense mechanism effectively restores and improves the model's accuracy.
- **Flipping Attack:**
  As shown in *Fig. 8(c)* and *Fig. 8(d)*, although the success rate of label-flipping attacks is inherently lower than that of backdoor attacks, the introduction of the PDFL scheme remains highly effective in counteracting them. Correspondingly, *Fig. 8(d)* demonstrate that PDFL also brings about a significant enhancement in the accuracy of the main federated learning task.

2) CIFAR10 Dataset: *Fig. 9* illustrates the effectiveness of the PDFL defenses against backdoor and flipping attacks on the CIFAR10 dataset.

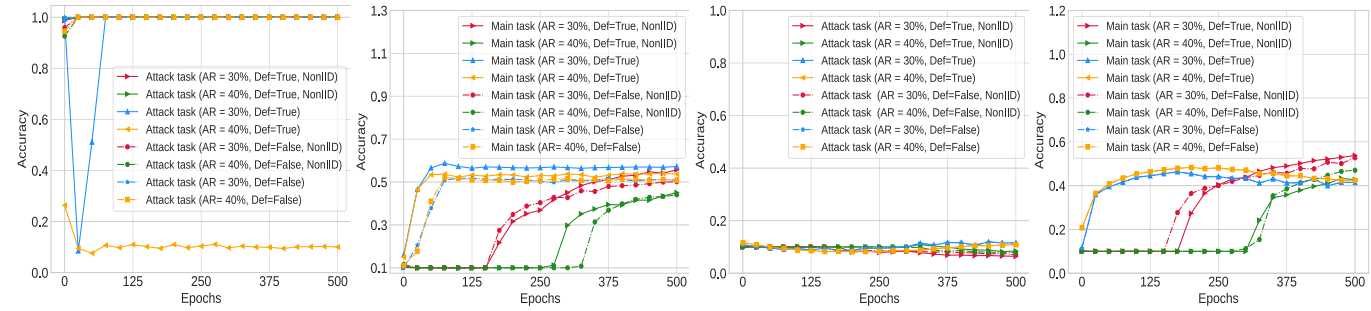
- **Backdoor Attack:**
  The attack success rate from the adversary's perspective is illustrated in *Fig. 9(a)*. Under a 40% attacker scenario, the PDFL dramatically reduced the success rate of backdoor attacks from nearly 1 to approximately 10%. Meanwhile, from the perspective of honest participants, *Fig. 9(b)* show that the implementation of the PDFL defense mechanism improves the federated learning task success rate by 5-10 percentage points.
- **Flipping Attack:**
  As shown in *Fig. 9(c)* and *Fig. 9(d)*, under flipping attack conditions, both the attack success rate and federated learning accuracy are generally low. However, the introduction of PDFL still demonstrates a significant effect:

Authorized licensed use limited to: Anhui University. Downloaded on October 18,2026 at 03:38:23 UTC from IEEE Xplore. Restrictions apply.



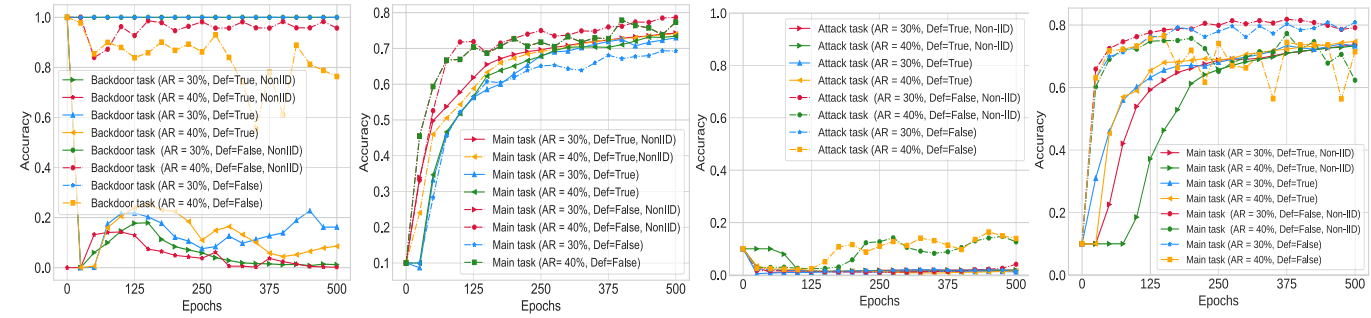
(a) Backdoor task (MNIST Backdoor) (b) Main task (MNIST Backdoor) (c) Flipping task (MNIST Flipping) (d) Main task (MNIST Flipping)

Fig. 8. Evaluation on MNIST Dataset.



(a) Backdoor task (CIFAR10 Backdoor) (b) Main task (CIFAR10 Backdoor) (c) Flipping task (CIFAR10 Flipping) (d) Main task (CIFAR10 Flipping)

Fig. 9. Evaluation on CIFAR10 Dataset.



(a) Backdoor task (FashionMNIST Backdoor) (b) Main task (FashionMNIST Backdoor) (c) Flipping task (FashionMNIST Flipping) (d) Main task (FashionMNIST Flipping)

Fig. 10. Evaluation on FashionMNIST Dataset.

it reduces the attack success rate while improving the modeling accuracy.

3) FashionMNIST Dataset: *Fig. 10* illustrates the effectiveness of the PDFL defenses against backdoor and flipping attacks on the FashionMINIST dataset.

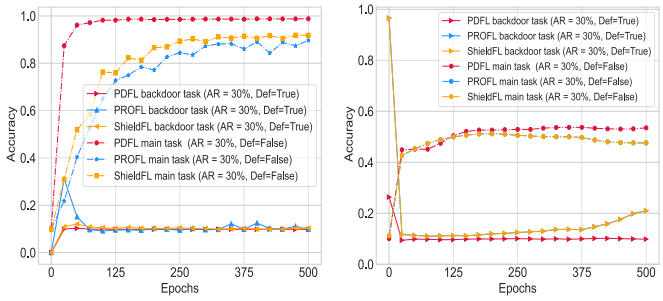
- **Backdoor Attack:**
  The attack success rate from the adversary's perspective is illustrated in *Fig. 10(a)*. Under 40%, 30% attacker scenarios, PDFL demonstrates effectiveness in defending against backdoor attacks under both NonIID and IID data distributions. Meanwhile, from the perspective of honest participants, *Fig. 10 (b)* show that the implementation of the PDFL defense mechanism improves the federated learning task success rate.
- **Flipping Attack:**
  As depicted in *Fig. 10(c)* and *Fig. 10(d)*, when facing flipping attacks, both the success rate of the attack and the accuracy of federated learning remain at relatively low levels overall. Nevertheless, the adoption of PDFL still exhibits a notable impact: it lowers the attack success rate and, at the same time, enhances the modeling accuracy.

E. Model Accuracy Comparison With Exist Work

We compare our PDFL with two state-of-the-art robustness FL, ie. ShieldFL [12] and PROFL [26]. The data is distributed to $m = 100$ clients, of which 30%, i.e. $n = 30$ are attackers.

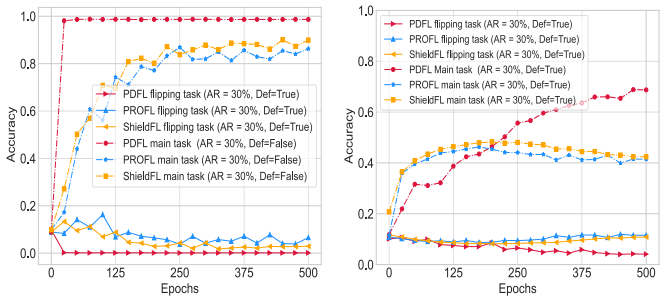
1) Backdoor Attack: *Fig. 11(a)* and *Fig. 11 (b)* show respectively the accuracy comparison of different defense mechanisms on the MNIST and CIFAR10 dataset.

Authorized licensed use limited to: Anhui University. Downloaded on October 18,2026 at 03:38:23 UTC from IEEE Xplore. Restrictions apply.



(a) MNIST dataset (b) CIFAR10 dataset

Fig. 11. Accuracy comparision with exist work defense backdoor attack.



(a) MINIST dataset (b) CIFAR10 dataset

Fig. 12. Accuracy comparison with exist work defense flipping attack.

All three models demonstrate significant effectiveness in resisting backdoor attacks. Notably, after several iterations, the PDFL scheme achieves the highest accuracy when defense mechanisms are implemented. This highlights PDFL's strong capability in countering backdoor threats. Nevertheless, it should be noted that PDFL may require additional training rounds to reach stability, as establishing reliable client identities necessitates learning across multiple iterations.

2) Label-Flipping Attack: *Fig. 12(a)* shows the accuracy comparison of different defense mechanisms on the MNIST dataset. Similarly, *Fig. 12(b)* presents the corresponding comparison on the CIFAR-10 dataset. The results on CIFAR-10 under label-flipping attacks are particularly noteworthy. Without any defense mechanisms in place, the PDFL model performs the worst under these attack conditions. This underscores the importance of implementing robust defense strategies to enhance model resilience against label-flipping attacks.

VII. Conclusion

In this paper, we propose a privacy-enhancing and robust poisoning defense federated learning scheme. The main objective of the scheme is to enhance data privacy and defend against poisoning attacks. In federated aggregation, we introduce a dual-server mechanism and the two-parties MPC algorithm to prevent the leakage of model parameters. Additionally, we have designed four privacy-preserving protocols: the SecJudge, the SecCosine, the SecClu and the SecAgg protocols under the dual server setup to defense against poisoning attacks. Experiments have shown that our method improves accuracy by 10% – 20% in comparing to existing methods in defending against poisoning attacks.

A. Limitations and Future Work

The present study focuses on a two-party computation (2PC) setting and has been validated primarily in relatively small-scale experimental scenarios. While these evaluations provide useful insights into the feasibility of combining privacy preservation with poisoning defense, the applicability to larger and more heterogeneous real-world environments remains to be further explored. As part of future work, we aim to extend the framework to more general multi-party computation (MPC) settings to improve scalability and resilience, investigate techniques to further enhance robustness and efficiency, and adapt the proposed protocols to transformer-based architectures with potential applications to large language models (LLMs).

References

[1] B. Li, P. Qi, and B. Liu, "Trustworthy AI: From principles to practices," *ACM Comput. Surveys*, vol. 55, no. 9, pp. 1–46, Sep. 2023.

[2] Z. Wang, J. Ma, X. Wang, J. Hu, Z. Qin, and K. Ren, "Threats to training: A survey of poisoning attacks and defenses on machine learning systems," *ACM Comput. Surveys*, vol. 55, no. 7, pp. 1–36, Jul. 2023.

[3] T. Qin, X. Gao, J. Zhao, K. Ye, and C.-Z. Xu, "APBench: A unified benchmark for availability poisoning attacks and defenses," 2023, *arXiv:2308.03258*.

[4] K.-Y. Lam, X. Lu, L. Zhang, X. Wang, H. Wang, and S. Q. Goh, "Efficient FHE-based privacy-enhanced neural network for trustworthy AI-as-a-service," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 5, pp. 4451–4468, Sep. 2024.

[5] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2017, pp. 3–18.

[6] P. Qiu et al., "Your labels are selling you out: Relation leaks in vertical federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 5, pp. 3653–3668, Sep. 2023.

[7] X. Lyu et al., "CoBA: Collusive backdoor attacks with optimized trigger to federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 2, pp. 1506–1518, Mar. 2025.

[8] Y. Bai, Y. Chen, and H. Zhang, "Villain: Backdoor attacks against vertical split learning," in *Proc. 32nd USENIX Secur. Symp.*, vol. 2023, Aug. 2023, pp. 2743–2760.

[9] Y. Xiao, X. Li, T. Li, R. Wang, Y. Pang, and G. Wang, "A distributed generative adversarial network for data augmentation under vertical federated learning," *IEEE Trans. Big Data*, vol. 11, no. 1, pp. 74–85, Feb. 2025.

[10] M. Abdullah, V. Bapu, K. Akash, A. Mannan Khan, and M. S. Bhargavi, "Label flipping attacks on federated learning: GAN-based poisoning and countermeasures," in *Proc. Int. Conf. Data Sci. Netw. Secur. (ICDSNS)*, Jul. 2024, pp. 1–6.

[11] P. Blanchard, E. M. El Mhamdi, and R. Guerraoui, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 118–128.

[12] Z. Ma, J. Ma, Y. Miao, Y. Li, and R. H. Deng, "ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 1639–1654, 2022.

[13] P. Qiu, X. Zhang, S. Ji, C. Fu, X. Yang, and T. Wang, "HashVFL: Defending against data reconstruction attacks in vertical federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 3435–3450, 2024.

[14] X. Chen, H. Yu, X. Jia, and X. Yu, "APFed: Anti-poisoning attacks in privacy-preserving heterogeneous federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5749–5761, 2023.

[15] J. Liu, X. Li, X. Liu, H. Zhang, Y. Miao, and R. H. Deng, "DefendFL: A privacy-preserving federated learning scheme against poisoning attacks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 5, pp. 9098–9111, May 2025.

[16] L. Shen et al., "SPEFL: Efficient security and privacy-enhanced federated learning against poisoning attacks," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 13437–13451, Apr. 2024.

Authorized licensed use limited to: Anhui University. Downloaded on October 18,2026 at 03:38:23 UTC from IEEE Xplore. Restrictions apply.

[17] G. Xia, J. Chen, X. Huang, C. Yu, and Z. Zhang, "FL-PTD: A privacy preserving defense strategy against poisoning attacks in federated learning," in *Proc. IEEE 47th Annu. Comput., Softw., Appl. Conf. (COMPSAC)*, Jun. 2023, pp. 735–740.
[18] Z. Lu, S. Lu, Y. Cui, X. Tang, and J. Wu, "Split aggregation: Lightweight privacy-preserving federated learning resistant to Byzantine attacks," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 5575–5590, 2024.
[19] X. Ma, H. Sun, R. Q. Hu, and Y. Qian, "A new implementation of federated learning for privacy and security enhancement," in *Proc. IEEE Global Commun. Conf.*, Dec. 2022, pp. 4885–4890.
[20] B. Deressa and M. A. Hasan, "TrustBandit: Optimizing client selection for robust federated learning against poisoning attacks," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Apr. 2024, pp. 1–8.
[21] X. Liu, H. Li, G. Xu, Z. Chen, X. Huang, and R. Lu, "Privacy-enhanced federated learning against poisoning adversaries," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 4574–4588, 2021.
[22] H. Lycklama, L. Burkhalter, A. Viand, N. Küchler, and A. Hithnawi, "RoFL: Robustness of secure federated learning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2023, pp. 453–476.
[23] B. Hitaj, G. Ateniese, and F. Perez-Cruz, "Deep models under the GAN: Information leakage from collaborative deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 603–618.
[24] Y. Miao et al., "RFed: Robustness-enhanced privacy-preserving federated learning against poisoning attack," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 5814–5827, 2024.
[25] X. Hao, C. Lin, W. Dong, X. Huang, and H. Xiong, "Robust and secure federated learning against hybrid attacks: A generic architecture," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 1576–1588, 2024.
[26] Y. Zhong and L.-P. Wang, "PROFL: A privacy-preserving federated learning method with stringent defense against poisoning attacks," in *Proc. 27th Int. Conf. Comput. Supported Cooperat. Work Design (CSCWD)*, May 2024, pp. 260–265.
[27] C. Xie, Y. Long, P.-Y. Chen, Q. Li, S. Koyejo, and B. Li, "Unraveling the connections between privacy and certified robustness in federated learning against poisoning attacks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2023, pp. 1511–1525.
[28] Y. Li, G. Wang, T. Wang, and W. Jia, "FL-HSDP: Client-side defense for federated learning via Hessian optimization and shuffled differential privacy," in *Proc. IEEE Int. Conf. High Perform. Comput. Commun., Data Sci. Syst., Smart City Dependability Sensor, Cloud Big Data Syst. Appl. (HPCC/DSS/SmartCity/DependSys)*, Dec. 2023, pp. 696–703.
[29] X. Li, X. Yang, Z. Zhou, and R. Lu, "Efficiently achieving privacy preservation and poisoning attack resistance in federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 4358–4373, 2024.
[30] Z. Chen, S. Yu, M. Fan, X. Liu, and R. H. Deng, "Privacy-enhancing and robust backdoor defense for federated learning on heterogeneous data," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 693–707, 2024.
[31] C. Dong et al., "Privacy-preserving and Byzantine-robust federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 2, pp. 889–904, Feb. 2023.
[32] T. Muhr and W. Zhang, "Privacy-preserving detection of poisoning attacks in federated learning," in *Proc. 19th Annu. Int. Conf. Privacy, Secur. Trust (PST)*, Aug. 2022, pp. 1–10.
[33] T. Schneider, A. Suresh, and H. Yalame, "Comments on 'privacy-enhanced federated learning against poisoning Adversarie,"' *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 1407–1409, 2023.
[34] F. Sattler, K.-R. Müller, T. Wiegand, and W. Samek, "On the Byzantine robustness of clustered federated learning," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2020, pp. 8861–8865.
[35] R. Fotohi, F. Shams Aliee, and B. Farahani, "A lightweight and secure deep learning model for privacy-preserving federated learning in intelligent enterprises," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 31988–31998, Oct. 2024, doi: 10.1109/JIOT.2024.3421602.
[36] R. Fotohi, F. Shams Aliee, and B. Farahani, "Decentralized and robust privacy-preserving model using blockchain-enabled federated deep learning in intelligent enterprises," *Appl. Soft Comput.*, vol. 161, Aug. 2024, Art. no. 111764, doi: 10.1016/j.asoc.2024.111764.
[37] M. Li, S. S. M. Chow, S. Hu, Y. Yan, C. Shen, and Q. Wang, "Optimizing privacy-preserving outsourced convolutional neural network predictions," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 1592–1604, May 2022.
[38] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.



Gang Wang received the B.Sc. degree in information security and the M.S. degree in cyberspace and information security from Xidian University, Xi'an, China, in 2018 and 2021, respectively. He joined Zhejiang Lab in 2022. He also joined Nanyang Technological University in 2024. His research interests include machine learning security and privacy preservation.



Huiwen Wu received the B.S. degree in mathematics from Sichuan University, Chengdu, China, in 2013, and the Ph.D. degree in mathematics from the University of California, USA, in 2019. From 2020 to 2022, she was a Senior Algorithm Engineer with the Ant Group, where she focused on the design and deployment of privacy-preserving machine learning algorithms in real-world systems. In 2022, she joined Zhejiang Lab as a Researcher. Her current research interests include privacy-preserving machine learning, privacy-enhancing techniques for large language models, large-scale stochastic optimization, and random matrix theory. She leads Zhejiang Lab Talent Fund for Young Professionals and has published 15 peer-reviewed conference papers. She has also served on the program committees of several leading venues, including AAAI, PRICAI, and INCRYPT.



Qingming Li received the B.S. degree from Xidian University, Xi'an, China, in 2017, and the Ph.D. degree from Tsinghua University, Beijing, China, in 2022. She was a Research Assistant with the Research Institute of Artificial Intelligence, Zhejiang Lab, Hangzhou, China, from July 2022 to March 2024. Since April 2024, she has been a Post-Doctoral Researcher with the College of Computer Science and Technology, Zhejiang University, Hangzhou. Her research interests include deep learning security, large language models, federated learning, and data valuation.



Ziyao Liu received the bachelor's degree from Zhengzhou University, China, in 2015, the master's degree from Beijing Institute of Technology, China, in 2018, and the Ph.D. degree in computer science from Nanyang Technological University, Singapore, in 2023. He is currently a Research Fellow with the National Centre for Research in Digital Trust (DTC), Singapore. His research interests include trustworthy AI, secure multi-party computation, and applied cryptography.

Authorized licensed use limited to: Anhui University. Downloaded on October 18,2026 at 03:38:23 UTC from IEEE Xplore. Restrictions apply.



Jun Zhao (Member, IEEE) received the bachelor's degree from Shanghai Jiao Tong University, China, in July 2010, and the Ph.D. degree in electrical and computer engineering from Carnegie Mellon University (CMU), USA, affiliating with CMU's renowned CyLab Security & Privacy Institute in May 2015. He is currently an Associate Professor with the School of Computer Science, Shanghai Jiao Tong University, China. He was an Assistant Professor with Nanyang Technological University (NTU), Singapore. Before joining NTU first as a Post-Doctoral Researcher and then as a Faculty Member, he was a Post-Doctoral Researcher with Arizona State University as an Arizona Computing Post-Doctoral Best Practices Fellow. He is currently an Editor of IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY and IEEE INTERNET OF THINGS JOURNAL. He was also selected as a Best Editor of IEEE WIRELESS COMMUNICATIONS LETTERS and IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING in 2023.



Kwok-Yan Lam (Senior Member, IEEE) received the B.Sc. degree (Hons.) from the University of London, London, U.K., in 1987, and the Ph.D. degree from the University of Cambridge, Cambridge, U.K., in 1990. He is currently the Associate Vice President (Strategy and Partnerships) and a Professor with the College of Computing and Data Science, Nanyang Technological University (NTU), Singapore. He is also the Executive Director of the Digital Trust Centre Singapore (DTC), which is a national center also designated the Singapore AI Safety Institute. Since August 2020, he has been on part-time secondment to the INTERPOL as a Consultant at Cyber and New Technology Innovation. He has been a Board Member of International Telecommunication Union (ITU)'s Academic Advisory Body on Emerging Technologies since September 2025. He has been a Professor with Tsinghua University, China (2002–2010) and a Faculty Member of the National University of Singapore and the University of London since 1990. He was a Visiting Scientist with the Isaac Newton Institute of the Cambridge University and a Visiting Professor with European Institute for Systems Security, Karlsruhe, Germany. His research interests include distributed systems, the IoT security infrastructure, cybersecurity, privacy-enhancing techniques, digital trust, AI safety, and quantum resilience. He received Singapore Foundation Award in 1998 from Japanese Chamber of Commerce and Industry in recognition of his research and development achievement in information security in Singapore. He was a recipient of the 2022 Singapore Cybersecurity Hall of Fame Award.



Qingkuan Dong received the B.S. degree in communications engineering and the M.S. and Ph.D. degrees in cryptology from Xidian University, Xi'an, China, in 1998, 2000, and 2004, respectively. From 2004 to 2005, he was with the Post-Doctoral Research Center, Institute of Software, Chinese Academy of Sciences, Beijing. He is currently an Associate Professor with the State Key Laboratory of Integrated Services Networks, Xidian University. His research interests include cryptology, the IoT security, information security, and trusted networks.



Yao Yang received the Ph.D. degree from the Department of Physics, Shanghai Jiao Tong University, in 2011. He is currently a Research Expert with Zhejiang Lab, China. He has published more than 30 research papers in top international journals and conferences. His research interests include deep representation learning, federated learning, and interdisciplinary topics of machine learning and science.



Li Zhou received the Ph.D. degree in statistics from the University of Lemans in 2013. She was a Lecturer with the School of Mathematics and Statistics, Shandong University, from 2013 to 2022. She joined subsequently Zhejiang Lab in 2022. Her research interests are probability and statistics, both on theory and applications, including statistics for diffusion process, privacy protecting, machine learning and their applications in finance, smart transportation, and data generation.

Authorized licensed use limited to: Anhui University. Downloaded on October 18,2026 at 03:38:23 UTC from IEEE Xplore. Restrictions apply.